

Chapter 5

VDMA Controller (VDMA)

5.1 Overview

DMA (Direct Memory Access) enables direct access to system memory or peripherals without CPU involvement. The VDMA controller on ESP32-P4 is a general-purpose DMA that performs high-speed data transfer from memory to memory, from memory to peripheral, and from peripheral to memory. The VDMA complies with the AXI3 protocol and includes two AXI master interfaces. This design allows users to select between the two interfaces for data transfer dynamically.

5.2 Terminology

The following terms are defined in the context of the VDMA controller on ESP32-P4:

- **AXI:** Advanced eXtensible Interface, a part of AMBA 3 (Advanced Microcontroller Bus Architecture) specification, designed to facilitate high bandwidth and low latency communication between memory and peripheral, and between memory and memory.
- **Peripheral:** A system component that has a handshake interface with VDMA, as listed in Table 5.4-1.
- **Memory:** A system component that is always ready for DMA transfer, which means there is no handshaking interface between VDMA and memory. The memory that supports VDMA data transfer includes HP L2MEM, external flash, and external RAM.
- **Source:** Source memory or peripheral from which VDMA reads data via the AXI interface. VDMA then stores data in channel FIFO.
- **Source transfer:** Transfer from the source to VDMA.
- **Destination:** Destination peripheral or memory to which VDMA writes the data stored in channel FIFO.
- **Destination transfer:** Transfer from VDMA to the destination.
- **Channel:** Data path between source and destination.
- **Master interface:** Refers to the two AXI master interfaces of VDMA.
- **Slave interface:** The APB3 slave interface for programming registers.
- **Handshaking interface:** The handshaking interface performs a handshake between VDMA and the source or destination peripheral. The handshaking interface requests, acknowledges, and controls an VDMA transfer.
- **Flow controller:** The flow controller defines the length of a DMA block transfer and terminates the transfer. The flow controller can be VDMA, a source peripheral, or a destination peripheral, but it cannot be memory.

5.3 Features

VDMA supports the following features:

- Four channels for unidirectional data transfer from source to destination
- Two AXI master interfaces
- Handshake with MIPI DSI (Display Serial Interface) and ISP (Image Signal Processor)
- Memory-to-memory, ISP-to-memory, and MIPI DSI-to-memory transfer types
- Multiple levels of DMA transfer hierarchy
- Configurable transfer type, transfer length, and transfer size for each channel
- Single-block transfer
- Multi-block transfer based on contiguous address, automatic reloading register configuration, shadow registers, and linked lists
- Independent configuration of multi-block transfer type for source transfer and destination transfer
- Channel disabling without data loss
- Channel suspension, resume, and abortion
- Configurable priorities among arbitration channels
- Flow control using VDMA or peripherals
- Programmable mapping between peripherals and channels

5.4 Architectural Overview

Figure [5.4-1](#) shows the architecture of VDMA and its connection with other system components.

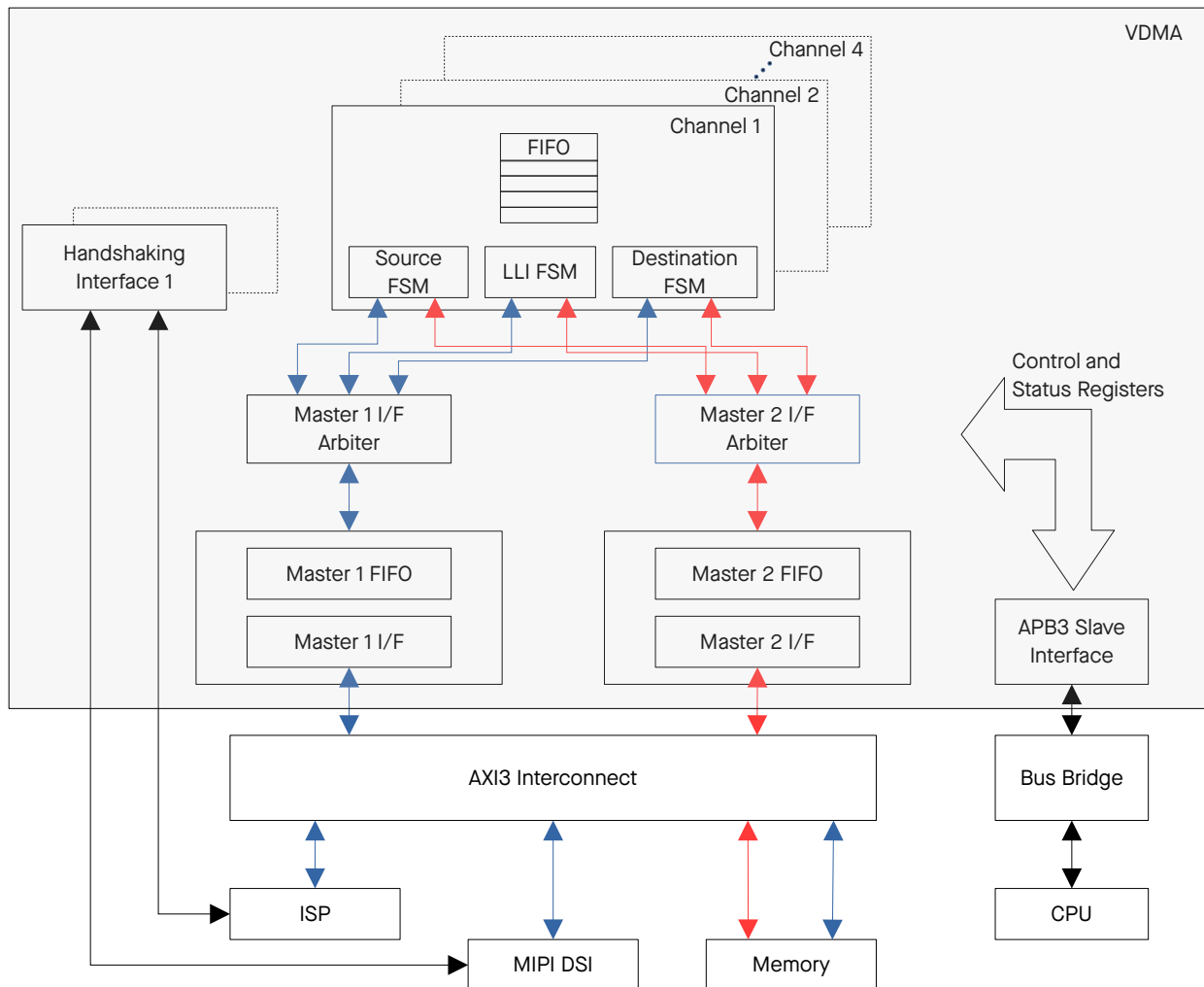


Figure 5.4-1. VDMA Architecture

It can be seen from the figure that:

- The CPU accesses VDMA registers via the APB3 slave interface.
- VDMA has four channels (Channel 1 to Channel 4). Each channel has a data FIFO with a depth of 64 and a width of 64 bits.
- VDMA has two AXI master interfaces (Master 1 and Master 2).
 - Master 1 can access ISP, MIPI DSI, and memory, as blue arrows indicate.
 - Master 2 can only access memory, as red arrows indicate.
 - Master 1 and Master 2 each have an arbiter that arbitrates read and write requests on all channels.
- VDMA has three hardware handshaking interfaces. Table 5.4-1 shows the mapping between the handshaking interfaces and peripherals.

Table 5.4-1. Hardware Handshaking Interface and Peripherals Mapping Table

Handshaking Interface Number	Peripheral
0	MIPI DSI

1	ISP (ISP-to-memory)
2	ISP (memory-to-ISP)

For more information about ISP and MIPI DSI, please refer to Chapter [36 Image Signal Processor \(ISP\)](#) and Chapter [4 MIPI DSI \[to be added later\]](#).

5.5 Functional Description

This section provides a detailed description of the key features and operations of the module.

5.5.1 Transfer Hierarchy

Transfers are organized into a maximum of four levels: DMA transfer level, block transfer level, transaction level, and AXI transfer level.

Figure [5.5-1](#) shows four transfer levels between VDMA and peripherals.

Figure [5.5-2](#) shows three transfer levels between VDMA and memory. There is no transaction level because memory is assumed to be always ready for data transfer.

The following descriptions of DMA, block, and AXI transfers apply to the transfer between VDMA and memory, VDMA and peripherals. **The description of the transaction level applies only to transfers between VDMA and peripherals.**

- **DMA transfer:** Software defines the number of blocks in a DMA transfer. Once a DMA transfer has finished, VDMA disables the channel and generates an interrupt. The channel can then be reprogrammed for a new DMA transfer. A DMA transfer can consist of either a single-block or a multi-block transfer.
- **Block transfer:** A block transfer moves a specified data block through the VDMA. The flow controller controls the block length. There are two types of block transfer: single-block transfer and multi-block transfer. As shown in Figure [5.5-1](#), a block transfer may consist of multiple transactions.
- **Transaction:** A peripheral request may trigger either a single or burst transaction, corresponding to an AXI single transfer or multiple AXI burst transfers, respectively.
 - Single transaction: The length of a single transaction is always 1, in the unit of `DMAC_CHn_SRC_TR_WIDTH`.
 - Burst transaction: The length of a burst transaction is programmed into the VDMA through `DMAC_CHn_SRC_MSIZE` for the source or `DMAC_CHn_DST_MSIZE` for the destination. The burst length typically relates to the FIFO sizes in the source or destination peripherals. For more information, please refer to the chapter on each peripheral.
- **AXI transfer:** Refers to the AXI protocol transfer, which is divided into AXI single transfer and AXI burst transfer.

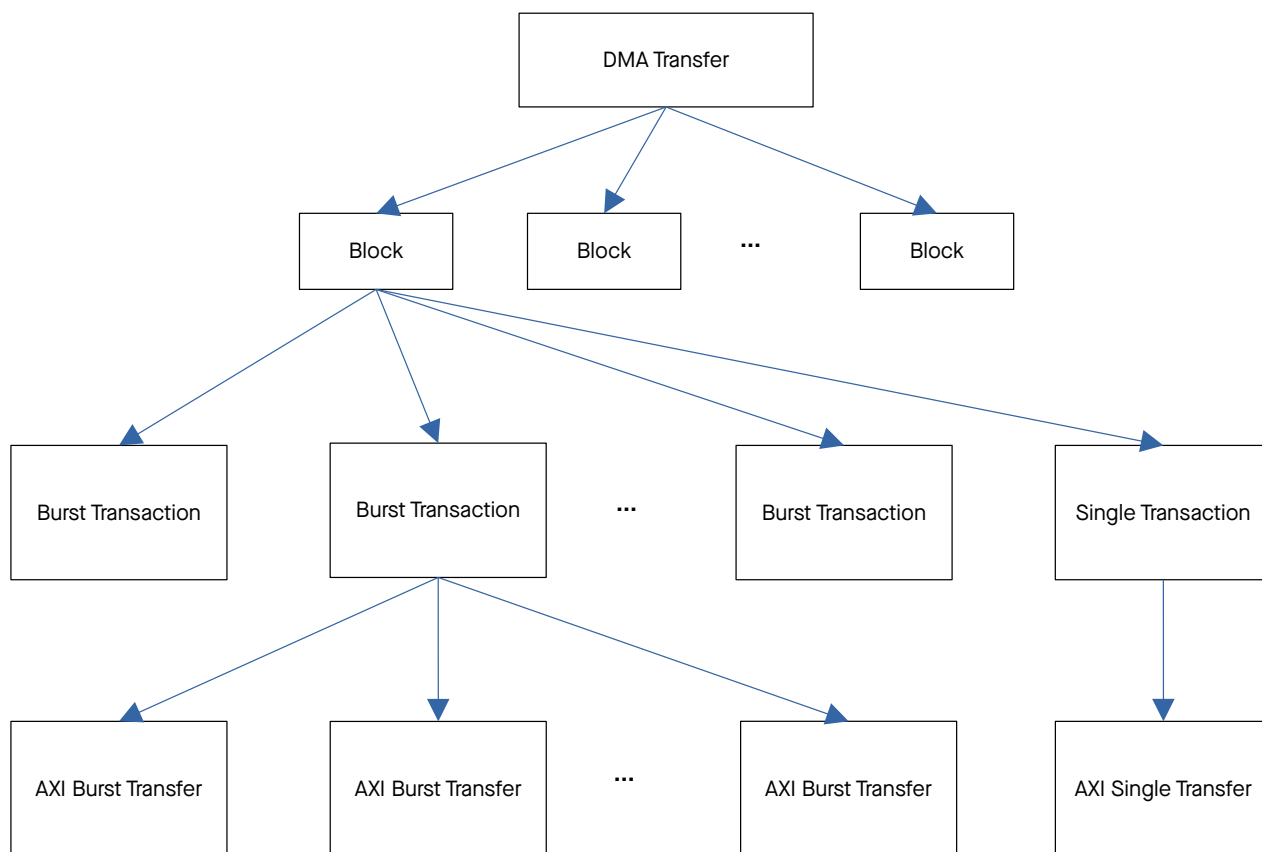


Figure 5.5-1. VDMA Transfer Hierarchy for Peripherals

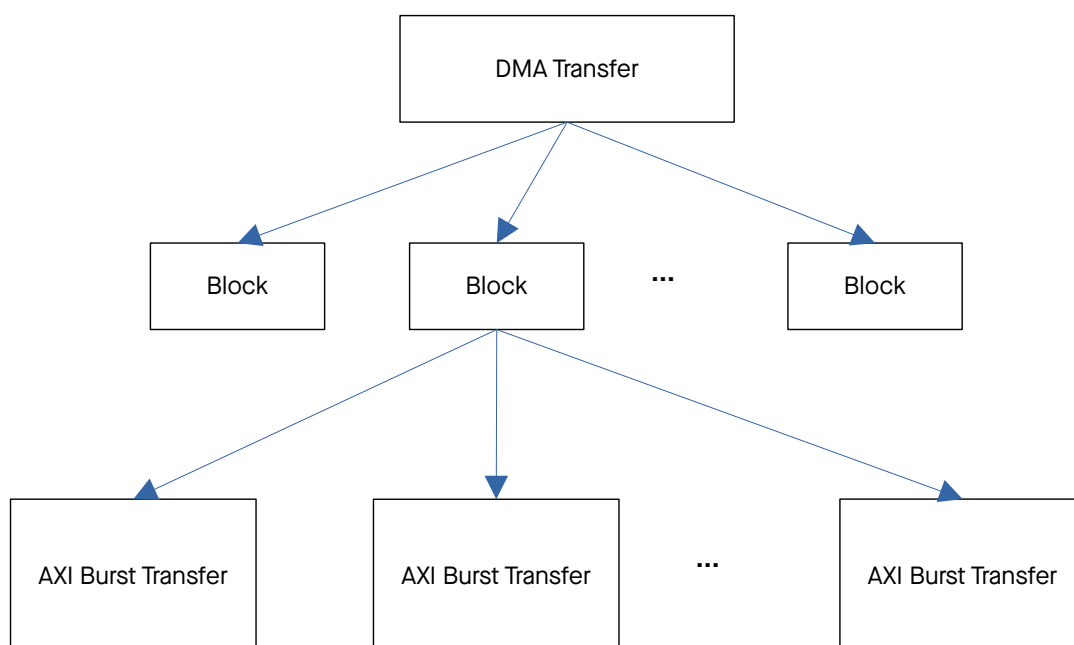


Figure 5.5-2. VDMA Transfer Hierarchy for Memory

5.5.2 Arbitration Scheme

The arbitration mechanism arbitrates read and write requests for all channels.

VDMA uses a priority and fair-among-equals arbitration mechanism. The priority for each channel is defined in the channel configuration register `DMAC_CH $n$ _CFG1_REG`.

A priority of 3 is the highest, while 0 is the lowest. Multiple channels can share the same priority level.

The arbitration mechanism works as follows:

- The arbitration mechanism determines which request client to grant the authorization signal based on the priority of the request. The request with the highest priority will be granted access to the channel.
- When two or more requests have the same priority, request clients will gain access randomly.

5.5.2.1 Read Arbiter

The read arbiter arbitrates read requests, including:

- VDMA's read requests to obtain data or status from the source
- VDMA's read requests to obtain status from the destination
- VDMA's read requests to obtain LLI descriptors

The block diagram of the read arbiter is shown in Figure 5.5-3:

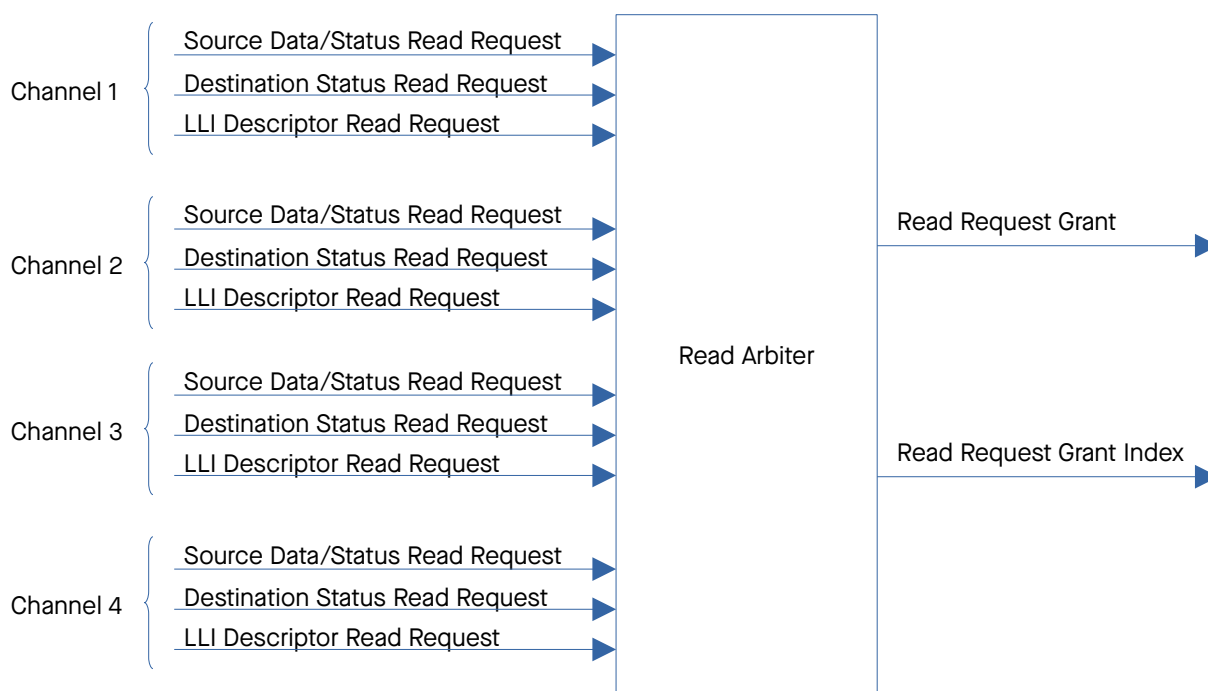


Figure 5.5-3. VDMA Read Arbiter

Read requests to obtain linked list descriptors will be granted the highest priority, regardless of the channels' priority settings.

5.5.2.2 Write Arbiter

The write arbiter arbitrates write requests, including:

- Destination Data Write Request
- LLI Descriptor Write-Back Request

The block diagram of the write arbiter is shown in Figure 5.5-4:

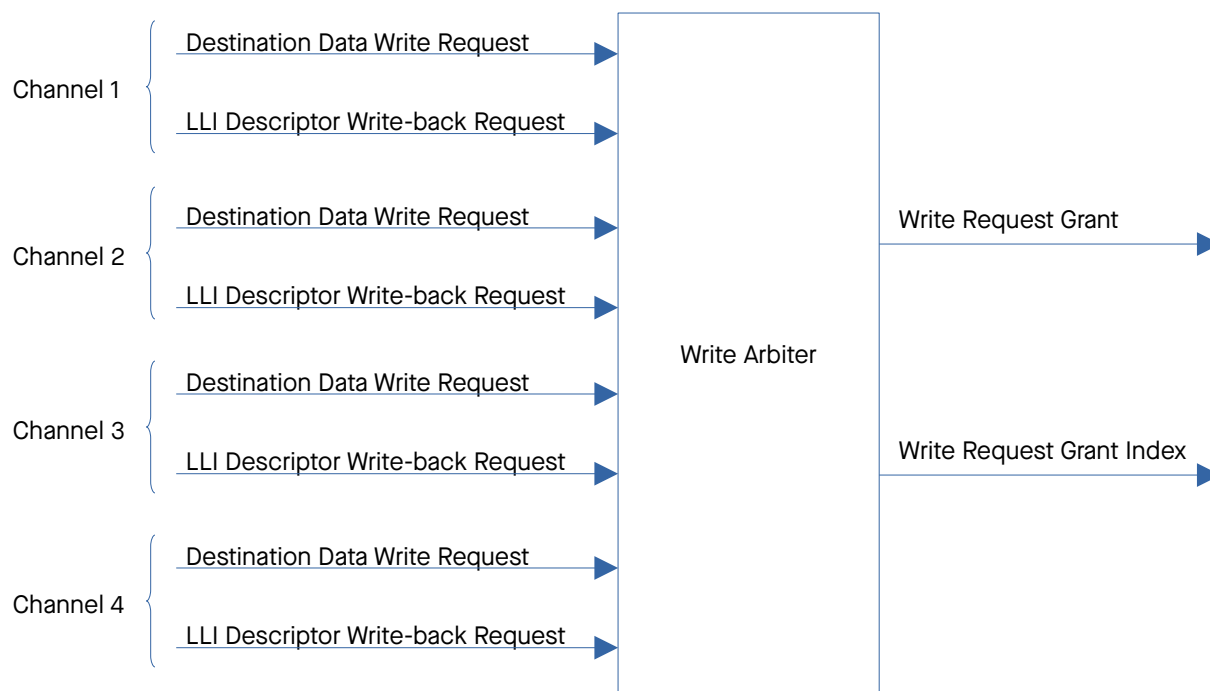


Figure 5.5-4. VDMA Write Arbiter

Within the same channel, Destination Data Write Request has higher priority than LLI Descriptor Write-Back Request. If different channels have the same priority, then Data Write Requests have the highest priority.

5.5.3 Handshaking Interface

A peripheral uses a handshaking interface to inform VDMA that it is ready to transmit or receive data via AXI bus. The operation of the handshaking interface depends on whether the flow controller is the peripheral or VDMA.

VDMA performs a single transaction or a burst transaction per handshake. In some cases, a block transfer cannot be completed using only burst transactions. This situation typically occurs when the block size is not a multiple of the burst transaction length. In this case, the block transfer uses burst transactions until the remaining size of the block is less than the amount of data in the burst transaction. At this point, a single transaction is used to complete the block transfer.

A source's burst transaction size is determined by `DMAC_CHn_SRC_MSIZ`. A destination's burst transaction size is determined by `DMAC_CHn_DST_MSIZ`. The transaction size also corresponds to the burst transaction size used for handshaking. The value of `DMAC_CHn_SRC_MSIZ` or `DMAC_CHn_DST_MSIZ` should match the peripheral configuration. For detailed information, please refer to the chapter on related peripherals.

VDMA has three hardware handshaking interfaces with MIPI DSI, ISP (ISP-to-memory), and ISP (memory-to-ISP). For the mapping relationship please see Table 5.4-1.

5.5.4 Transfer Control

Transfer control logic facilitates the data transfer from a source to a destination. Data from the source is temporarily stored in the channel FIFO before being sent to the destination. If source and destination peripherals use different transfer sizes, VDMA will pack and unpack the data to fit the FIFO configuration.

For a specific DMA transfer, the transfer type and flow control configurations are determined by `DMAC_CHn_TT_FC`.

5.5.4.1 Single-Block Transfer

If a DMA transfer consists of a single block, the software can set `DMAC_CHn_SRC_MULTBLK_TYPE` and `DMAC_CHn_DST_MULTBLK_TYPE` to 0 to enable contiguous-address-based single-block transfer. In this case, VDMA disables the channel once the block transfer of size `DMAC_CHn_BLOCK_TS` is completed.

Note:

Single-block transfer is a special case of multi-block transfer, and there is no strict distinction between the two. In this chapter, single-block transfer refers to the transfer consisting of only one block, while multi-block transfer refers to the transfer consisting of at least two blocks.

5.5.4.2 Multi-Block Transfer

If a DMA transfer consists of multiple blocks, the software can configure `DMAC_CHn_SRC_MULTBLK_TYPE` and `DMAC_CHn_DST_MULTBLK_TYPE` to choose a multi-block transfer type. There are four types of multi-block transfers, depending on how the transfer control registers (`DMAC_CHn_SARO_REG`, `DMAC_CHn_DARO_REG`, `DMAC_CHn_BLOCK_TS`, `DMAC_CHn_CTLO/1_REG`) are updated:

1. **Contiguous address**
2. **Auto reloading:** Register values reload from initial values
3. **Shadow register:** Register values load from shadow registers
4. **Linked list** Register values load from the next linked list (LLI)

Table 5.5-1 lists all the cases of register update methods for multi-block transfer.

Table 5.5-1. Register Update Methods for Multi-Block Transfer

DMAC_CH n _CFG0_REG		Registers Update Method				Comment
DMAC_CH n _SRC_MULTBLK_TYPE	DMAC_CH n _DST_MULTBLK_TYPE	DMAC_CH n _SARO_REG	DMAC_CH n _DARO_REG	DMAC_CH n _CTLO_REG / CTL1_REG	DMAC_CH n _BLOCK_TS	
00	00	Contiguous address	Contiguous address	No update	No update	Single block or last block of multi-block
00	01	Contiguous address	Reload from initial value	Reload from initial value	Reload from initial value	-
00	10	Contiguous address	Load from shadow register	Load from shadow register	Load from shadow register	-
01	00	Reload from initial value	Contiguous address	Reload from initial value	Reload from initial value	-
01	01	Reload from initial value	Reload from initial value	Reload from initial value	Reload from initial value	-
01	10	Reload from initial value	Load from shadow register	Load from shadow register	Load from shadow register	-
10	00	Load from shadow register	Contiguous address	Load from shadow register	Load from shadow register	-
10	01	Load from shadow register	Reload from initial value	Load from shadow register	Load from shadow register	-
10	10	Load from shadow register	Load from shadow register	Load from shadow register	Load from shadow register	-
00	11	Contiguous address	Load from next LLI	Load from next LLI	Load from next LLI	-
01	11	Reload from initial value	Load from next LLI	Load from next LLI	Load from next LLI	-
11	00	Load from next LLI	Contiguous address	Load from next LLI	Load from next LLI	-
11	01	Load from next LLI	Reload from initial value	Load from next LLI	Load from next LLI	-
11	11	Load from next LLI	Load from next LLI	Load from next LLI	Load from next LLI	-
10	11	-	-	-	-	Invalid. Interrupt* is generated.
11	10	-	-	-	-	Invalid. Interrupt* is generated.

*Note: The interrupt is [DMAC_CH \$n\$ _SLVIF_MULTIBLKTYPE_ERR_INT](#).

Contiguous Address

In this scenario, the address for each subsequent block continues from the end of the preceding block. To achieve this contiguity, configure `DMAC_CHn_SRC_MULTBLK_TYPE` or `DMAC_CHn_DST_MULTBLK_TYPE` to 0 for the source address or destination address to remain continuous across the blocks.

In the case of multi-block transfer, the values of `DMAC_CHn_SRC_MULTBLK_TYPE` and `DMAC_CHn_DST_MULTBLK_TYPE` cannot both be selected as contiguous addresses, meaning they cannot both be configured to 0 at the same time. If both are configured to 0 during a multi-block transfer, then the current block will be considered as the last block.

During a multi-block transfer, if `DMAC_CHn_SARO_REG` and `DMAC_CHn_DARO_REG` need to have contiguous addresses between blocks, you can indirectly achieve this functionality using linked-list or shadow-register-based multi-block transfer.

Note:

If you have enabled contiguous-address-based multi-block transfer (i.e., `DMAC_CHn_SRC_MULTBLK_TYPE` or `DMAC_CHn_DST_MULTBLK_TYPE` is 0), ensure there are at least two blocks in the DMA transfer. Otherwise, it will result in unpredictable behavior.

Auto Reloading

In this scenario, `DMAC_CHn_SARO_REG` and `DMAC_CHn_DARO_REG` are reloaded with their initial values at the end of each block. VDMA does not proceed to the next block transfer until software clears the corresponding channel's block transfer complete interrupt by writing 1 to `DMAC_CHn_CLEAR_SRC_TRANSCOMP_INTSTAT` and `DMAC_CHn_CLEAR_DST_TRANSCOMP_INTSTAT`.

Note:

If you have enabled auto-reloading-based multi-block transfer, please ensure there are at least two blocks in the DMA transfer. Otherwise, it will result in unpredictable behavior.

Shadow Register

In this scenario, `DMAC_CHn_SARO_REG`, `DMAC_CHn_DARO_REG`, `DMAC_CHn_BLOCK_TS`, `DMAC_CHn_CTLO_REG`, and `DMAC_CHn_CTL1_REG` synchronize with their corresponding shadow registers when each block finishes. The values of the shadow registers are then used for the subsequent block.

Software writes the block configurations to the corresponding shadow registers. VDMA copies the shadow register contents to `DMAC_CHn_SARO_REG`, `DMAC_CHn_DARO_REG`, `DMAC_CHn_BLOCK_TS`, `DMAC_CHn_CTLO_REG`, and `DMAC_CHn_CTL1_REG` before starting the subsequent block transfer.

Read operations to the `DMAC_CHn_SARO_REG`, `DMAC_CHn_DARO_REG`, `DMAC_CHn_BLOCK_TS`, `DMAC_CHn_CTLO_REG`, and `DMAC_CHn_CTL1_REG` always return the data corresponding to the current block transfer, not the shadow register contents that correspond to the next block.

`DMAC_CHn_SHADOWREG_OR_LLI_VALID` indicates whether the shadow register contents are valid (1) or not (0). If this bit reads as 0 during a shadow register fetch phase, VDMA discards the shadow register contents and generates a `DMAC_CHn_SHADOWREG_OR_LLI_INVALID_ERR_INT` interrupt. VDMA waits until the software

writes 1 to [DMAC_CHn_BLK_TFR_RESUMEREQ](#), indicating that the shadow registers are available. After this, VDMA attempts to read the shadow registers again and continues with the next block transfer.

Note:

In a shadow-register-based multi-block transfer, the values in [DMAC_CHn_SARO_REG](#), [DMAC_CHn_DARO_REG](#), [DMAC_CHn_BLOCK_TS](#), [DMAC_CHn_CTLO_REG](#), and [DMAC_CHn_CTL1_REG](#) correspond to the current transfer configurations. When software writes to these registers, it is actually writing to the corresponding shadow registers.

Linked List

In this scenario, VDMA retrieves a block descriptor for a specific block from memory to update [DMAC_CHn_SARO_REG](#), [DMAC_CHn_DARO_REG](#), [DMAC_CHn_BLOCK_TS](#), [DMAC_CHn_CTLO_REG](#), and [DMAC_CHn_CTL1_REG](#), and then initiates the block transfer. This process is called LLI (Linked List Item) update. The block chaining feature of VDMA uses the linked list pointer register [DMAC_CHn_LOCO](#) to store the address of the next linked list item in memory, thereby achieving seamless continuity of data transfer.

Figure 5.5-5 shows the VDMA linked list item:

31		0
Reserved		
Reserved		
CHn_LL_P_STATUS[63:32]		
CHn_LL_P_STATUS[31:0]		
Write Back for CHn_DSTAT		
Write Back for CHn_SSTAT		
CHn_CTL1[31:0]		
CHn_CTLO[31:0]		
Reserved		
CHn_LLPO[31:6]		Reserved[5:0]
Reserved		
CHn_BLOCK_TS[31:0]		
Reserved		
CHn_DAR[31:0]		
Reserved		
CHn_SAR[31:0]		

Figure 5.5-5. VDMA Linked List Item (Descriptor)

VDMA can dynamically extend linked lists, which means you do not need to pre-create the entire linked list in system memory. To enable this feature, set [DMAC_CHn_SHADOWREG_OR_LLI_VALID](#) to 1. If the linked list item is the last one, then also configure [DMAC_CHn_SHADOWREG_OR_LLI_LAST](#) to 1.

DMAC_CH n _SHADOWREG_OR_LLI_VALID indicates whether the linked list item fetched from memory is valid (1) or not (0). If the LLI is invalid, VDMA discards the LLI and generates a **DMAC_CH n _SHADOWREG_OR_LLI_INVALID_ERR_INT** interrupt. This interrupt causes VDMA to halt the corresponding channel and wait until the software writes 1 to **DMAC_CH n _BLK_TFR_RESUMEREQ** to indicate that the LLI is available. After this, VDMA will attempt to read the LLI again.

Note:

For pre-fetched LLIs, if **DMAC_CH n _SHADOWREG_OR_LLI_VALID** is 0, then the **DMAC_CH n _SHADOWREG_OR_LLI_INVALID_ERR_INTSTAT** interrupt will not be generated. VDMA re-attempts to fetch the LLI again after completing the current block transfer and generates a **DMAC_CH n _SHADOWREG_OR_LLI_INVALID_ERR_INTSTAT** interrupt only if **DMAC_CH n _SHADOWREG_OR_LLI_VALID** still reads as 0.

The following fields in LLI are prepared by software:

- **CH n _SAR**: The field definition is the same as **DMAC_CH n _SARO_REG**
- **CH n _DAR**: The field definition is the same as **DMAC_CH n _DARO_REG**
- **CH n _BLOCK_TS**: The field definition is the same as **DMAC_CH n _BLOCK_TS**
- **CH n _LLPO[31:6]**: The field definition is the same as **DMAC_CH n _LLPO_REG[31:6]**
- **CH n _CTL0**: The field definition is the same as **DMAC_CH n _CTL0_REG**
- **CH n _CTL1**: The field definition is the same as **DMAC_CH n _CTL1_REG**

If the status write-back option is enabled, VDMA writes back **CH n _CTL0**, **CH n _CTL1**, **CH n _LLP_STATUS**, **CH n _SSTAT**, and **CH n _DSTAT** information to the location defined for this field, which is from address **[CH n _LLP] + 0x20** to **[CH n _LLP] + 0x34**. The write-back of **CH n _SSTAT** and **CH n _DSTAT** can be independently enabled or disabled by programming **DMAC_CH n _SRC_STAT_EN** and **DMAC_CH n _DST_STAT_EN**.

Note:

- If **CH n _SSTAT** and **CH n _DSTAT** write-back is not enabled, do not use PSRAM to store LLI.
- **DMAC_CH n _SHADOWREG_OR_LLI_VALID** will be 0 after the LLI write-back operation.

Figure 5.5-6 shows **CH n _LLP_STATUS** write-back field of LLI:

63				0
*	Reserved[61:47]	Data Left in Channel FIFO (CH n _Status[46:32])	Reserved[31:22]	Completed Block Transfer Size (CH n _Status[21:0])

Note: * denotes Status Indication (CH n _IntStatus[1:0])

Figure 5.5-6. CH n _LLP_STATUS Write-Back Field of LLI

CH n _LLP_STATUS[63] and **CH n _LLP_STATUS[62]** indicate **DMA_TFR_DONE** and **BLOCK_TFR_DONE** status respectively. These two fields are the last to be updated during LLI write-back. Software should ensure that **BLOCK_TFR_DONE** bit is set to 1 before using **CH n _SSTAT** and **CH n _DSTAT** information. **DMA_TFR_DONE** and **BLOCK_TFR_DONE** bits will be set to 1 after transferring the last block.

CH n _STATUS[21:0] corresponds to [DMAC_CH \$n\$ _CMPLTD_BLK_TFR_SIZE](#), and CH n _STATUS[46:32] corresponds to [DMAC_CH \$n\$ _DATA_LEFT_IN_FIFO\[14:0\]](#).

Suspension of Transfers Between Blocks

At the end of every block transfer, a block transfer completion interrupt [DMAC_CH \$n\$ _BLOCK_TFR_DONE_INT](#) is generated if:

- Global interrupt is enabled ([DMAC_INT_EN](#) = 1)
- The channel block transfer completion interrupt is enabled
([DMAC_CH \$n\$ _ENABLE_BLOCK_TFR_DONE_INTSTAT](#) = 1,
[DMAC_CH \$n\$ _ENABLE_BLOCK_TFR_DONE_INTSIGNAL](#) = 1, [DMAC_CH \$n\$ _IOC_BLK_TFR](#) = 1)

For contiguous-address and auto-reloading-based multi-block transfers (neither source nor destination peripheral uses shadow-register or linked-list-based multi-block transfers), the DMA transfer automatically stalls after the [DMAC_CH \$n\$ _BLOCK_TFR_DONE_INT](#) interrupt is generated. VDMA does not proceed to the next block transfer until software writes 1 to [DMAC_CH \$n\$ _CLEAR_BLOCK_TFR_DONE_INTSTAT](#) to clear the interrupt.

Channel suspension between blocks ensures that the block transfer done ISR (Interrupt Service Routine) of the next-to-last block is serviced before the final block transfer starts. This ensures that the ISR has cleared [DMAC_CH \$n\$ _SRC_MULTBLK_TYPE](#) and [DMAC_CH \$n\$ _DST_MULTBLK_TYPE](#) before the final block transfer is completed.

End of Multi-Block Transfers

If either source or destination peripheral uses shadow-register or linked-list-based multi-block transfers, then [DMAC_CH \$n\$ _SHADOWREG_OR_LLI_LAST](#) indicates whether the current block is the last in the transfer. If this bit is 1, VDMA recognizes that the current block is the final block in the transfer and completes the DMA transfer operation at the end of the current block transfer.

For contiguous-address and auto-reloading-based multi-block transfers (when neither source nor destination peripheral uses shadow-register or linked-list-based multi-block transfers), if the corresponding multi-block type selection fields [DMAC_CH \$n\$ _SRC_MULTBLK_TYPE](#) and/or [DMAC_CH \$n\$ _DST_MULTBLK_TYPE](#) are 0 at the end of a block transfer, VDMA understands that the previous block was the final block in the transfer and completes the DMA transfer operation.

5.5.5 Flow Controller

The flow controller determines the length of the DMA block transfer. The flow controller can be VDMA, the source peripheral, or the destination peripheral; memory cannot serve as a flow controller.

If the block length is known before enabling the channel, configure VDMA as the flow controller. If the block length is unknown before enabling the channel, use the source or destination peripheral as the flow controller to determine when to terminate the DMA block transfer.

5.5.6 Channel Suspend and Resume

Software can suspend and resume a channel during DMA transfer. To suspend a channel during DMA transfer:

1. Software writes 1 to `DMAC_CHn_SUSP`.
2. VDMA halts all transfers from the source, after completing all AXI transfers initiated from the source.
3. VDMA sets `DMAC_CHn_CH_SRC_SUSPENDED_INT` to 1 to indicate that source transfer is suspended.
4. VDMA transfers all the data in channel FIFO to the destination. If `DMAC_CHn_SRC_TR_WIDTH < DMAC_CHn_DST_TR_WIDTH` and `DMAC_CHn_SUSP` is 1, there may still be data in the channel FIFO, but not enough to form a single transfer of `DMAC_CHn_DST_TR_WIDTH`. The remaining data in the channel FIFO will be transferred to the destination if the channel is resumed.
5. VDMA sets `DMAC_CHn_CH_SUSPENDED_INT` to 1 to indicate that the channel is suspended.

After a channel suspend, software can resume the channel by setting `DMAC_CHn_SUSP` to 0. Then, VDMA resumes the DMA transfer from the point where it was suspended.

5.5.7 Channel Disable

Under normal operation, software enables a channel by writing 1 to `DMAC_CHn_EN`, and VDMA clears `DMAC_CHn_EN` after channel transfer completion. Software can disable a channel before a transfer completes.

5.5.7.1 Disabling a Suspended Channel Before Transfer Completion

To disable a suspended channel before transfer completion:

- Write 0 to `DMAC_CHn_EN` after `DMAC_CHn_CH_SUSPENDED_INTSTAT` is set to 1. If `DMAC_CHn_SRC_TR_WIDTH < DMAC_CHn_DST_TR_WIDTH` and `DMAC_CHn_SUSP` is 1, there may still be data in the channel FIFO, but not enough to form a single transfer of `DMAC_CHn_DST_TR_WIDTH`. In this case, once the channel is disabled, the remaining data in the channel FIFO is not transferred to the destination peripheral and is lost.
- VDMA sets `DMAC_CHn_CH_DISABLED_INTSTAT` to 1 to indicate that the channel is disabled and generates an interrupt.

5.5.7.2 Disabling a Non-suspended Channel Before Transfer Completion

To disable a non-suspended channel before transfer completion:

- Software writes 0 to `DMAC_CHn_EN` during DMA transfer.
- VDMA halts all transfers from the source, after completing all AXI transfers initiated on the source.
- VDMA transfers all the data in the channel FIFO to the destination. When `DMAC_CHn_SRC_TR_WIDTH < DMAC_CHn_DST_TR_WIDTH` and `DMAC_CHn_EN` is 0, there may still be data in the channel FIFO, but not enough to form a single transfer of `DMAC_CHn_DST_TR_WIDTH`. In this scenario, once the channel is disabled, the remaining data in the channel FIFO is not transferred to the destination peripheral and is lost.
- VDMA sets `DMAC_CHn_CH_DISABLED_INTSTAT` to 1 to indicate that the channel is disabled and generates an interrupt.

Note:

Once software initiates a channel disable procedure by writing 0 to `DMAC_CHn_EN`, before `DMAC_CHn_CH_DISABLED_INT` is set to 1, you cannot re-enable the channel by writing 1 to `DMAC_CHn_EN`. VDMA will ignore this write operation.

5.5.8 Low-Power Technique

VDMA implements a low-power technique that intelligently monitors idle periods across multiple DMA channels, the slave bus interface, and AXI master interface channels in various scenarios. By detecting inactivity, this feature can turn off the clock for these modules, significantly reducing overall power consumption.

When activity is detected, the logic quickly restores the clocks to these modules, ensuring that performance remains uncompromised.

Configure `DMAC_LOWPOWER_CFGO_REG` to enable the low-power feature and manage the low-power counters.

5.5.8.1 Low-Power Technique for DMA Channels

DMA channels occupy a significant portion of the chip area in the VDMA module, resulting in higher power consumption compared to other components. Therefore, optimizing power usage in this module is essential.

The low-power technique for DMA channels employs a **DMA channel low-power state machine** to monitor activities. When the state machine detects a low-power condition, it activates a **DMA channel low-power delay counter**. This counter runs until it expires. Once the counter expires, the DMA channels enter a low-power state by gating the `AXI_DMAC_CORE_CLK`.

The DMA channels will remain in this low-power state until any activity is detected. If activity is detected while the delay counter is still running, the counter is reset and paused until the low-power condition is detected again.

5.5.8.2 Low-Power Technique for Slave Bus Interface

The slave bus interface unit (SBIU) accesses the internal registers of VDMA via the APB3 slave interface. The low-power technique can optimize the power consumption in SBIU.

The low-power technique for the slave bus interface employs a **SBIU state machine** to monitor activities. When a low-power condition is detected (i.e., when there are no ongoing APB reads or writes and the SBIU state machine is idle), the **SBIU low power delay counter** is activated. Once the delay counter expires, SBIU enters the low-power state by gating `PCLK` or `AXI_DMAC_CORE_CLK` (based on the clock mode configuration).

SBIU remains in the low-power state until an active read or write transaction is detected on the slave bus interface. If an active transaction is detected while the SBIU low-power delay counter running, the delay counter is reset and paused until the low-power condition is detected again.

If a read or write transaction occurs while SBIU is in a low-power state, the low-power logic immediately exits the low-power state by un-gating `PCLK` or `AXI_DMAC_CORE_CLK`.

5.5.8.3 Low-Power Technique for AXI Master Interface Channels

The AXI master interface module implements the AXI bus to transfer data between memory and peripheral.

Whenever a low-power condition is detected on the AXI master interface channels, the **AXI master interface channel low-power delay counter** starts. The AXI low-power logic waits until the delay counter expires. After the delay counter expires, the AXI master interface channel enters the low-power state by gating the clock of AXI master interface and AXI_DMAMCORE_CLK. It remains in this state until any activity is detected. If an activity is detected while the AXI master interface channel low-power delay counter is running, the delay counter is reset and paused until the low-power condition is detected again.

5.5.8.4 Global Low-Power Technique

The global low-power technique implements the low-power option based on the idle condition of the DMA channel, slave bus interface, and AXI master interface channels. If all the mentioned modules indicate that the VDMA is idle, the **global low-power delay counter** is activated.

The global low-power logic waits until the low-power delay counter expires. After the counter expires, VDMA enters the low-power state by gating the clocks of all modules. It remains in the low-power state until any activity is detected on any of the modules. If an activity is detected while the global low-power delay counter is running, the global delay counter is reset and paused until the low-power condition is again detected.

5.6 Interrupts

ESP32-P4's VDMA can generate the interrupt signal AXI_DMAM_INTR that will be sent to the [Interrupt Matrix](#).

The interrupt signal is generated by the internal interrupt sources from VDMA.

- Common interrupts:
 - DMAM_SLVIF_UNDEINEDREG_DEC_ERR_INT: Slave interface undefined register decode error interrupt
 - DMAM_SLVIF_COMMONREG_RD2WO_ERR_INT: Slave interface common register read to write-only error interrupt
 - DMAM_SLVIF_COMMONREG_WR2RO_ERR_INT: Slave interface common register write to read-only error Interrupt
 - DMAM_SLVIF_COMMONREG_DEC_ERR_INT: Slave interface common register decode error interrupt
- Channel interrupts ($n = 1 \sim 4$):
 - DMAM_CH n _CH_ABORTED_INT: Channel aborted interrupt
 - DMAM_CH n _CH_DISABLED_INT: Channel disabled interrupt
 - DMAM_CH n _CH_SUSPENDED_INT: Channel suspended interrupt
 - DMAM_CH n _CH_SRC_SUSPENDED_INT: Channel source suspended interrupt
 - DMAM_CH n _SLVIF_SHADOWREG_WRON_VALID_ERR_INT: Shadow register write on valid error interrupt

- DMAC_CH n _SLVIF_WRONCHEN_ERR_INT: Slave interface write on enabled channel error interrupt
- DMAC_CH n _SLVIF_RD2RWO_ERR_INT: Slave interface read to write-only error interrupt
- DMAC_CH n _SLVIF_WR2RO_ERR_INT: Slave interface write to read-only error interrupt
- DMAC_CH n _SLVIF_DEC_ERR_INT: Slave interface decode error interrupt
- DMAC_CH n _SLVIF_MULTIBLKTYPE_ERR_INT: Slave interface multi-block type error interrupt
- DMAC_CH n _SHADOWREG_OR_LLI_INVALID_ERR_INT: Shadow register or LLI invalid error interrupt
- DMAC_CH n _LLI_WR_SLV_ERR_INT: LLI write slave error interrupt
- DMAC_CH n _LLI_RD_SLV_ERR_INT: LLI read slave error interrupt
- DMAC_CH n _LLI_WR_DEC_ERR_INT: LLI write decode error interrupt
- DMAC_CH n _LLI_RD_DEC_ERR_INT: LLI read decode error interrupt
- DMAC_CH n _DST_SLV_ERR_INT: Destination slave error interrupt
- DMAC_CH n _SRC_SLV_ERR_INT: Source slave error interrupt
- DMAC_CH n _DST_DEC_ERR_INT: Destination decode error interrupt
- DMAC_CH n _SRC_DEC_ERR_INT: Source decode error interrupt
- DMAC_CH n _DST_TRANSCOMP_INT: Destination transfer completed interrupt
- DMAC_CH n _SRC_TRANSCOMP_INT: Source transfer completed interrupt
- DMAC_CH n _DMA_TFR_DONE_INT: DMA transfer done interrupt
- DMAC_CH n _BLOCK_TFR_DONE_INT: Block transfer done interrupt

5.7 Programming Procedures

This section outlines the programming steps for different types of transfers.

5.7.1 Common Programming Procedures

The following two steps are common and should be performed before other programming procedures.

1. Set [HP_SYS_CLKRST_GDMA_SYS_CLK_EN](#) to 1 to enable VDMA bus clock.
2. Configure [TEE_DMA_GDMA_CH \$n\$ _W_PMS](#) and [TEE_DMA_GDMA_CH \$n\$ _R_PMS](#) to define the access permission of VDMA. For more details please refer to Chapter [19 Permission Control \(PMS\)](#).

5.7.2 Programming Procedures for Shadow-Register-Based Multi-Block Transfer

1. Read [DMAC_CH \$n\$ _EN](#) to select an available (unused) channel.
2. Configure [DMAC_CH \$n\$ _SRC_MULTBLK_TYPE](#) and/or [DMAC_CH \$n\$ _DST_MULTBLK_TYPE](#) to 2 to enable shadow-register-based multi-block transfer for source transfer and/or destination transfer.

3. Configure `DMAC_CHn_SARO_REG` and/or `DMAC_CHn_DARO_REG`, `DMAC_CHn_BLOCK_TS`, `DMAC_CHn_CTLO_REG`, and `DMAC_CHn_CTL1_REG` for the first block. VDMA loads the configured values to the corresponding shadow registers.

Notice: The `DMAC_CHn_SHADOWREG_OR_LLI_VALID` field in `DMAC_CHn_CTL1_REG` must be the last one to be set to 1.

4. Set `DMAC_CHn_EN` to 1 to enable the corresponding channel.
5. VDMA initiates the DMA block transfer operation based on the configurations.
 - The block transfer might start immediately or after the handshaking request, depending on `DMAC_CHn_TT_FC`. Specifically,
 - When the source is memory, the source transfer starts immediately; when the destination is memory, the destination transfer starts immediately.
 - When the source is a peripheral, obtaining data from the source requires waiting for a handshake with the source. When the destination is a peripheral, writing data to the destination requires waiting for a handshake with the destination.
 - VDMA checks `DMAC_CHn_SHADOWREG_OR_LLI_VALID` and if it is seen as 0, VDMA waits until software writes 1 to `DMAC_CHn_BLK_TFR_RESUMEREQ` to indicate valid shadow registers availability. Then VDMA re-attempts shadow register fetch operation. In this case, VDMA may generate a `DMAC_CHn_SHADOWREG_OR_LLI_INVALID_ERR_INT` interrupt.
 - VDMA checks `DMAC_CHn_SHADOWREG_OR_LLI_VALID` and if it is seen as 1, VDMA copies the shadow register contents to the registers used for executing the DMA block transfer (i.e., `DMAC_CHn_SARO_REG` and/or `DMAC_CHn_DARO_REG`, `DMAC_CHn_BLOCK_TS`, `DMAC_CHn_CTLO_REG`, and `DMAC_CHn_CTL1_REG`) and clears `DMAC_CHn_SHADOWREG_OR_LLI_VALID`.
 - If VDMA sees `DMAC_CHn_SHADOWREG_OR_LLI_LAST` as 1, it understands that the current block is the final block in the transfer and completes the DMA transfer operation at the end of current block transfer.
 - If VDMA sees `DMAC_CHn_SHADOWREG_OR_LLI_LAST` as 0, it understands that there are one or more blocks to be transferred and checks `DMAC_CHn_SHADOWREG_OR_LLI_VALID` again at the end of current block transfer.

6. Software waits `DMAC_CHn_SHADOWREG_OR_LLI_VALID` to be cleared to 0.
7. Configure `DMAC_CHn_SARO_REG` and/or `DMAC_CHn_DARO_REG`, `DMAC_CHn_BLOCK_TS`, `DMAC_CHn_CTLO_REG`, and `DMAC_CHn_CTL1_REG` for the next block. If the next block is the last block, set `DMAC_CHn_SHADOWREG_OR_LLI_LAST` to 1.

Notice: `DMAC_CHn_SHADOWREG_OR_LLI_VALID` must be the last to be set to 1.

8. Software waits for a `DMAC_CHn_BLOCK_TFR_DONE_INT` interrupt or polls `DMAC_CHn_BLOCK_TFR_DONE_INTSTAT` until it reads as 1 and goes back to Step 6.

Note:

In cases where a `DMAC_CHn_SHADOWREG_OR_LLI_INVALID_ERR_INT` interrupt is generated, follow the recommended flow to resume transfer:

- Configure `DMAC_CHn_SARO_REG` and/or `DMAC_CHn_DARO_REG`, `DMAC_CHn_BLOCK_TS`, `DMAC_CHn_CTLO_REG`, and `DMAC_CHn_CTL1_REG` for the next block.
- Set `DMAC_CHn_CLEAR_SHADOWREG_OR_LLI_INVALID_ERR_INTSTAT` to 1 to clear the interrupt.
- Set `DMAC_CHn_BLK_TFR_RESUMEREQ` to 1 to request for resuming block transfer.

5.7.3 Programming Procedures for Linked-List-Based Multi-Block Transfer

1. Reads `DMAC_CHn_EN` to select an available (unused) channel.
2. Configure `DMAC_CHn_SRC_MULTBLK_TYPE` and/or `DMAC_CHn_DST_MULTBLK_TYPE` to 3 to enable linked-list-based transfer for source transfer and/or destination transfer.
3. Configure `DMAC_CHn_LLPO_REG` to define the base address of the first linked list item and the master interface on which the linked list item is available.
4. Create one or more linked list items in the system memory. You can create the entire linked list item in advance or dynamically extend it using `DMAC_CHn_SHADOWREG_OR_LLI_VALID` and `DMAC_CHn_SHADOWREG_OR_LLI_LAST` fields of the LLI. Follow the steps below to dynamically extend the linked list.
 - During the DMA transfer process, when the linked list is not ready, configure `DMAC_CHn_SHADOWREG_OR_LLI_VALID` in the linked list to 0 and wait until the linked list is prepared. Then set `DMAC_CHn_SHADOWREG_OR_LLI_VALID` to 1 to extend the linked list.
 - When it is time to end the DMA transfer, set `DMAC_CHn_SHADOWREG_OR_LLI_LAST` of the last linked list to 1 to indicate the end of the transfer.
5. Set `DMAC_CHn_EN` to 1 to enable the channel.

Note:

You can swap the sequence of Step 4 and Step 5. However, if Step 5 is performed before Step 4, or if the linked list item for the next block transfer is not available in system memory during the multi-block transfer (i.e., `DMAC_CHn_SHADOWREG_OR_LLI_VALID` of the fetched LLI is 0), VDMA may generate a `DMAC_CHn_SHADOWREG_OR_LLI_INVALID_ERR_INT` interrupt.

6. VDMA initiates the DMA block transfer operation based on the configurations. The block transfer might start immediately or after the handshaking request, depending on `DMAC_CHn_TT_FC`. VDMA copies the linked list contents to the registers used for executing the DMA block transfer (i.e., `DMAC_CHn_SARO_REG` and/or `DMAC_CHn_DARO_REG`, `DMAC_CHn_BLOCK_TS`, `DMAC_CHn_CTLO_REG`, and `DMAC_CHn_CTL1_REG`) and initiates the DMA block transfer.
7. During the linked list fetch phase:
 - If VDMA reads `DMAC_CHn_SHADOWREG_OR_LLI_LAST` as 1, it understands that the current block is the final block in the transfer and completes the DMA transfer operation at the end of the current block transfer.
 - If VDMA reads `DMAC_CHn_SHADOWREG_OR_LLI_LAST` as 0, it understands that there are one or more blocks to be transferred and goes to Step 6.

- If VDMA reads `DMAC_CH $n$ _SHADOWREG_OR_LLI_VALID` as 0, it may generate a `DMAC_CH $n$ _SHADOWREG_OR_LLI_INVALID_ERR_INT` interrupt. VDMA waits until software writes 1 to `DMAC_CH $n$ _BLK_TFR_RESUMEREQ` to indicate valid LLI availability before re-attempting the LLI read operation.

5.7.4 Programming Procedures for Single-Block Transfer

1. Read `DMAC_CH $n$ _EN` to choose a free (unused) channel.
2. Configure `DMAC_CH $n$ _SRC_MULTBLK_TYPE` and `DMAC_CH $n$ _DST_MULTBLK_TYPE` to 0 to enable contiguous-address-based single-block transfer.
3. Configure `DMAC_CH $n$ _SARO_REG` and/or `DMAC_CH $n$ _DARO_REG`, `DMAC_CH $n$ _BLOCK_TS`, `DMAC_CH $n$ _CTL0_REG`, and `DMAC_CH $n$ _CTL1_REG` for the block.
4. Set `DMAC_CH $n$ _EN` to 1 to enable the channel.
5. Software waits for the `DMAC_CH $n$ _BLOCK_TFR_DONE_INT` interrupt or polls `DMAC_CH $n$ _BLOCK_TFR_DONE_INTSTAT` until it is 1.

5.8 Register Summary

The addresses in this section are relative to the VDMA base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Version Control Registers			
DMAC_IDO_REG	VDMA ID number	0x0000	RO
DMAC_COMPVERO_REG	VDMA version number	0x0008	RO
Configuration Registers			
DMAC_CFGO_REG	VDMA Configuration register	0x0010	R/W
DMAC_CHENO_REG	VDMA channel enable register	0x0018	varies
DMAC_RESETO_REG	VDMA reset control register	0x0058	R/W
DMAC_LOWPOWER_CFGO_REG	VDMA low power configuration register 0	0x0060	R/W
DMAC_LOWPOWER_CFG1_REG	VDMA low power configuration register 1	0x0064	R/W
DMAC_CH1_SARO_REG	VDMA channel 1 source address register	0x0100	R/W
DMAC_CH1_DARO_REG	VDMA channel 1 destination address register	0x0108	R/W
DMAC_CH1_BLOCK_TSO_REG	VDMA channel 1 block transfer size register	0x0110	R/W
DMAC_CH1_CTLO_REG	VDMA channel 1 control register 0	0x0118	R/W
DMAC_CH1_CTL1_REG	VDMA channel 1 control register 1	0x011C	R/W
DMAC_CH1_CFGO_REG	VDMA channel 1 configuration register 0	0x0120	varies
DMAC_CH1_CFG1_REG	VDMA channel 1 configuration register 1	0x0124	varies
DMAC_CH1_LLPO_REG	VDMA channel 1 linked list pointer register	0x0128	R/W
DMAC_CH1_BLK_TFR_RESUMEREQO_REG	VDMA channel 1 block transfer resume request register	0x0148	WO
DMAC_CH1_AXI_IDO_REG	VDMA channel 1 AXI ID register	0x0150	R/W
DMAC_CH2_SARO_REG	VDMA channel 2 source address register	0x0200	R/W
DMAC_CH2_DARO_REG	VDMA channel 2 destination address register	0x0208	R/W
DMAC_CH2_BLOCK_TSO_REG	VDMA channel 2 block transfer size register	0x0210	R/W
DMAC_CH2_CTLO_REG	VDMA channel 2 control register 0	0x0218	R/W
DMAC_CH2_CTL1_REG	VDMA channel 2 control register 1	0x021C	R/W

Name	Description	Address	Access
DMAC_CH2_CFG0_REG	VDMA channel 2 configuration register 0	0x0220	varies
DMAC_CH2_CFG1_REG	VDMA channel 2 configuration register 1	0x0224	varies
DMAC_CH2_LLPO_REG	VDMA channel 2 linked list pointer register	0x0228	R/W
DMAC_CH2_BLK_TFR_RESUMEREQO_REG	VDMA channel 2 block transfer resume request register	0x0248	WO
DMAC_CH2_AXI_IDO_REG	VDMA channel 2 AXI ID register	0x0250	R/W
DMAC_CH3_SARO_REG	VDMA channel 3 source address register	0x0300	R/W
DMAC_CH3_DARO_REG	VDMA channel 3 destination address register	0x0308	R/W
DMAC_CH3_BLOCK_TSO_REG	VDMA channel 3 block transfer size register	0x0310	R/W
DMAC_CH3_CTLO_REG	VDMA channel 3 control register 0	0x0318	R/W
DMAC_CH3_CTL1_REG	VDMA channel 3 control register 1	0x031C	R/W
DMAC_CH3_CFG0_REG	VDMA channel 3 configuration register 0	0x0320	varies
DMAC_CH3_CFG1_REG	VDMA channel 3 configuration register 1	0x0324	varies
DMAC_CH3_LLPO_REG	VDMA channel 3 linked list pointer register	0x0328	R/W
DMAC_CH3_BLK_TFR_RESUMEREQO_REG	VDMA channel 3 block transfer resume request register	0x0348	WO
DMAC_CH3_AXI_IDO_REG	VDMA channel 3 AXI ID register	0x0350	R/W
DMAC_CH4_SARO_REG	VDMA channel 4 source address register	0x0400	R/W
DMAC_CH4_DARO_REG	VDMA channel 4 destination address register	0x0408	R/W
DMAC_CH4_BLOCK_TSO_REG	VDMA channel 4 block transfer size register	0x0410	R/W
DMAC_CH4_CTLO_REG	VDMA channel 4 control register 0	0x0418	R/W
DMAC_CH4_CTL1_REG	VDMA channel 4 control register 1	0x041C	R/W
DMAC_CH4_CFG0_REG	VDMA channel 4 configuration register 0	0x0420	varies
DMAC_CH4_CFG1_REG	VDMA channel 4 configuration register 1	0x0424	varies
DMAC_CH4_LLPO_REG	VDMA channel 4 linked list pointer register	0x0428	R/W
DMAC_CH4_BLK_TFR_RESUMEREQO_REG	VDMA channel 4 block transfer resume request register	0x0448	WO
DMAC_CH4_AXI_IDO_REG	VDMA channel 4 AXI ID register	0x0450	R/W
Interrupt Registers			

Name	Description	Address	Access
DMAC_INTSTATUSO_REG	VDMA interrupt status register	0x0030	RO
DMAC_COMMONREG_INTCLEARO_REG	VDMA common interrupt clear register	0x0038	WO
DMAC_COMMONREG_INTSTATUS_ENABLEO_REG	VDMA common interrupt status enable register	0x0040	varies
DMAC_COMMONREG_INTSIGNAL_ENABLEO_REG	VDMA common interrupt signal enable register	0x0048	varies
DMAC_COMMONREG_INTSTATUSO_REG	VDMA common interrupt status register	0x0050	RO
DMAC_CH1_INTSTATUS_ENABLEO_REG	VDMA channel 1 interrupt status enable register	0x0180	varies
DMAC_CH1_INTSTATUSO_REG	VDMA channel 1 interrupt status register	0x0188	RO
DMAC_CH1_INTSIGNAL_ENABLEO_REG	VDMA channel 1 interrupt signal enable register	0x0190	varies
DMAC_CH1_INTCLEARO_REG	VDMA channel 1 interrupt clear register	0x0198	WO
DMAC_CH2_INTSTATUS_ENABLEO_REG	VDMA channel 2 interrupt status enable register	0x0280	varies
DMAC_CH2_INTSTATUSO_REG	VDMA channel 2 interrupt status register	0x0288	RO
DMAC_CH2_INTSIGNAL_ENABLEO_REG	VDMA channel 2 interrupt signal enable register	0x0290	varies
DMAC_CH2_INTCLEARO_REG	VDMA channel 2 interrupt clear register	0x0298	WO
DMAC_CH3_INTSTATUS_ENABLEO_REG	VDMA channel 3 interrupt status enable register	0x0380	varies
DMAC_CH3_INTSTATUSO_REG	VDMA channel 3 interrupt status register	0x0388	RO
DMAC_CH3_INTSIGNAL_ENABLEO_REG	VDMA channel 3 interrupt signal enable register	0x0390	varies
DMAC_CH3_INTCLEARO_REG	VDMA channel 3 interrupt clear register	0x0398	WO
DMAC_CH4_INTSTATUS_ENABLEO_REG	VDMA channel 4 interrupt status enable register	0x0480	varies
DMAC_CH4_INTSTATUSO_REG	VDMA channel 4 interrupt status register	0x0488	RO
DMAC_CH4_INTSIGNAL_ENABLEO_REG	VDMA channel 4 interrupt signal enable register	0x0490	varies
DMAC_CH4_INTCLEARO_REG	VDMA channel 4 interrupt clear register	0x0498	WO
status registers			
DMAC_CH1_STATUSO_REG	VDMA channel 1 status register 0	0x0130	RO
DMAC_CH1_STATUS1_REG	VDMA channel 1 status register 1	0x0134	RO
DMAC_CH1_SSTATO_REG	VDMA channel 1 source status register	0x0160	RO
DMAC_CH1_DSTATO_REG	VDMA channel 1 destination status register	0x0168	RO
DMAC_CH1_SSTATARO_REG	VDMA channel 1 source status address register	0x0170	R/W

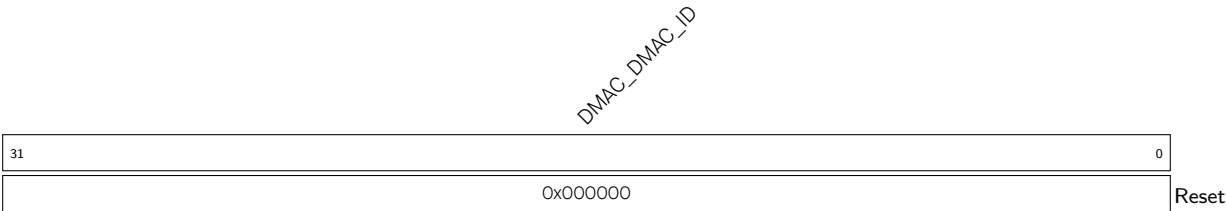
Name	Description	Address	Access
DMAC_CH1_DSTATARO_REG	VDMA channel 1 destination status address register	0x0178	R/W
DMAC_CH2_STATUSO_REG	VDMA channel 2 status register 0	0x0230	RO
DMAC_CH2_STATUS1_REG	VDMA channel 2 status register 1	0x0234	RO
DMAC_CH2_SSTATO_REG	VDMA channel 2 source status register	0x0260	RO
DMAC_CH2_DSTATO_REG	VDMA channel 2 destination status register	0x0268	RO
DMAC_CH2_SSTATARO_REG	VDMA channel 2 source status address register	0x0270	R/W
DMAC_CH2_DSTATARO_REG	VDMA channel 2 destination status address register	0x0278	R/W
DMAC_CH3_STATUSO_REG	VDMA channel 3 status register 0	0x0330	RO
DMAC_CH3_STATUS1_REG	VDMA channel 3 status register 1	0x0334	RO
DMAC_CH3_SSTATO_REG	VDMA channel 3 source status register	0x0360	RO
DMAC_CH3_DSTATO_REG	VDMA channel 3 destination status register	0x0368	RO
DMAC_CH3_SSTATARO_REG	VDMA channel 3 source status address register	0x0370	R/W
DMAC_CH3_DSTATARO_REG	VDMA channel 3 destination status address register	0x0378	R/W
DMAC_CH4_STATUSO_REG	VDMA channel 4 status register 0	0x0430	RO
DMAC_CH4_STATUS1_REG	VDMA channel 4 status register 1	0x0434	RO
DMAC_CH4_SSTATO_REG	VDMA channel 4 source status register	0x0460	RO
DMAC_CH4_DSTATO_REG	VDMA channel 4 destination status register	0x0468	RO
DMAC_CH4_SSTATARO_REG	VDMA channel 4 source status address register	0x0470	R/W
DMAC_CH4_DSTATARO_REG	VDMA channel 4 destination status address register	0x0478	R/W

5.9 Registers

The addresses in this section are relative to the VDMA base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

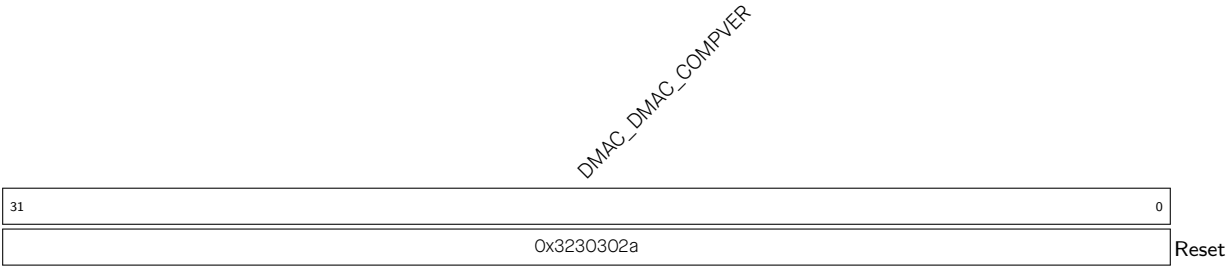
For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 5.1. DMAC_IDO_REG (0x0000)



DMAC_DMACH_ID Represents VDMA ID number. (RO)

Register 5.2. DMAC_COMPVERO_REG (0x0008)



DMAC_DMACH_COMPVER Represents VDMA version number. (RO)

Register 5.3. DMAC_CFG0_REG (0x0010)

(reserved)																															DMAC_INT_EN		DMAC_DMACE_EN																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																														
31																															2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																														
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

DMAC_DMACE_EN Configures whether to enable VDMA.

0: Disable

1: Enable

Note: If this field is cleared but a channel is still active, this field will still return 1. When hardware has terminated activity on all channels, this field will return 0.

(R/W)

DMAC_INT_EN Configures whether to enable global interrupt.

0: Disable

1: Enable

(R/W)

Register 5.4. DMAC_CHENO_REG (0x0018)

[illegible]

DMAC_CH n _EN (n : 1-4) Configures whether to enable VDMA channel n . Hardware automatically clears this bit after the last AXI transfer of the DMA transfer has been completed. Software can poll this field to determine when this channel is free for a new DMA transfer.

0: Disable channel *n*

1: Enable channel n

(R/W)

DMAC_CH*n*_EN_WE (*n*: 1-4) Configures whether to enable write to **DMAC_CH*n*_EN**. This field always reads as 0.

0: Disable write to `DMAC_CHn_EN`

1: Enable write to DMAC_CHn_EN

(WO)

DMAC_CH n _SUSP (n : 1-4) Configures whether to suspend channel n .

0: Do not suspend channel n

1: Suspend channel n

Software can clear this field to 0 after VDMA sets `DMAC_CHn_CH_SUSPENDED_INTSTAT` to 1, to exit the channel suspend mode.

Note: This field is cleared when channel *n* is disabled.

(R/W)

DMAC_CH n _SUSP_WE (n : 1-4) Configures whether to enable write to **DMAC_CH n _SUSP**. This field always reads as 0.

0: Disable write to DMAC_CHn_SUSP

1: Enable write to DMAC CH_n SUSP

(WO)

Register 5.5. DMAC_RESETO_REG (0x0058)

(reserved)																																DMAC_DMAR_RST			
31																															1	0	Reset		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	0

DMAC_DMAC_RST Configures whether to reset VDMA. Software writes 1 to this field to reset VDMA and polls this field to see if it is 0. VDMA resets all the modules except the APB slave interface module and clears this field to 0.

Note: Software is not allowed to write 0 to this field.

(R/W)

Register 5.6. DMAC_LOWPOWER_CFG0_REG (0x0060)

(reserved)																																DMAC_MMIF_CSLP_EN				DMAC_SBIU_CSLP_EN				DMAC_CHNL_CSLP_EN				DMAC_GBL_CSLP_EN			
31																															4	3	2	1	0												
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	Reset														

DMAC_GBL_CSLP_EN Configures whether to enable global low-power feature.

0: Disable

1: Enable

(R/W)

DMAC_CHNL_CSLP_EN Configures whether to enable low-power feature for DMA channels.

0: Disable

1: Enable

(R/W)

DMAC_SBIU_CSLP_EN SBIU Configures whether to enable low-power feature for slave bus interface.

0: Disable

1: Enable

(R/W)

DMAC_MXIF_CSLP_EN Configures whether to enable low-power feature for AXI master interface.

0: Disable

1: Enable

(R/W)

Register 5.7. DMAC_LOWPOWER_CFG1_REG (0x0064)

(reserved)								DMAC_MMIF_LPDLY								DMAC_SBIU_LPDLY								DMAC_GLCH_LPDLY								
3124								2316								158								70								
00000000								0x40								0x40								0x40								Reset

DMAC_GLCH_LPDLY Configures the value of the AXI master interface low-power delay counter. The programmed value must be greater than or equal to 0x4. If it is less than 0x4, the field will reset to 0x40. (R/W)

DMAC_SBIU_LPDLY Configures the value of the SBIU low-power delay counter. The programmed value must be greater than or equal to 0x4. If it is less than 0x4, the field will reset to 0x40.
(R/W)

DMAC_MXIF_LPDLY Configures the value of the global low-power delay counter and the DMA channel low-power delay counter. The programmed value must be greater than or equal to 0x4. If it is less than 0x4, the field will reset to 0x40. (R/W)

Register 5.8. DMAC_CH*n*_SARO_REG (*n*: 1-4) (0x0100**n*)

Diagram illustrating the DMAC register structure. The register is 32 bits wide, with bit 31 on the left and bit 0 on the right. The register is divided into two main sections: a 31-bit field on the left and a 1-bit field on the right. The 31-bit field is labeled "DMAC_CH" in red and "SAR0" in black. The 1-bit field is labeled "Reset".

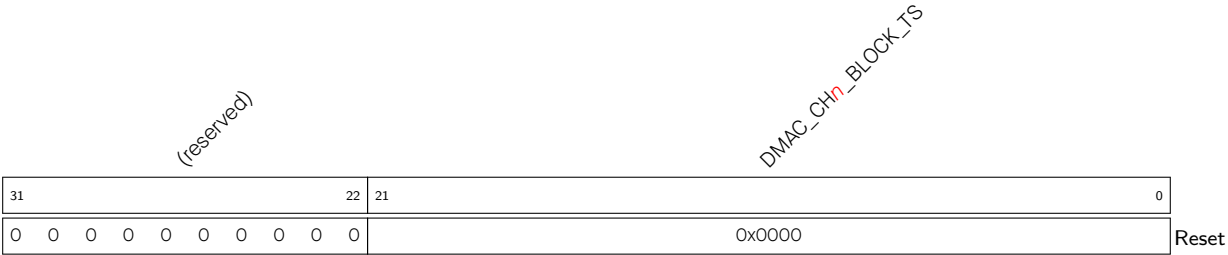
DMAC_CHn_SAR0 Configures the source address of DMA transfer. This field is updated after each source transfer. **DMAC_CHn_SINC** determines whether the address increments or remains unchanged on every source transfer throughout the block transfer. (R/W)

Register 5.9. DMAC_CH n _DARO_REG (n : 1-4) (0x0100* n + 0x0008)

Diagram of the DMAC_CHn_DAR0 register. The register is 32 bits wide, with bit 31 on the left and bit 0 on the right. The value 0x000000 is shown in the center, and the word 'Reset' is at the bottom right.

DMAC_CH n _DARO Configures the destination address of DMA transfer. This field is updated after each source transfer. **DMAC_CH n _DINC** determines whether the address increments or remains unchanged on every destination transfer throughout the block transfer. (R/W)

Register 5.10. DMAC_CHn_BLOCK_TSO_REG (n: 1-4) (0x0100*n + 0x0010)



DMAC_CHn_BLOCK_TS Configures the total number of **DMAC_CHn_SRC_TR_WIDTH** in a DMA transfer. The programming range of this field is 0 ~ 4,194,303.
Block transfer size = Programmed value + 1
(R/W)

Register 5.11. DMAC_CH n _CTLO_REG (n : 1-4) (0x0100* n + 0x0018)

(reserved)																															DMAC_CH _n _NONPOSTED_LASTWRITE_EN																															(reserved)																															DMAC_CH _n _DST_MSIZ																															DMAC_CH _n _SRC_MSIZ																															DMAC_CH _n _DST_TR_WIDTH																															DMAC_CH _n _SRC_TR_WIDTH																															(reserved)																															DMAC_CH _n _DINC																															(reserved)																															DMAC_CH _n _SINC																															(reserved)																															DMAC_CH _n _DMS																															DMAC_CH _n _SMS																														
31	30	29																					22	21											18	17											14	13											11	10											8	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																		
0	0	0																				0x0										0x0										0x2										0x2										0	0	0	0	0	0	0	0	0	0	Reset																																																																																																																																																																																																																																																																																																																																																																									

DMAC_CH n _SMS Configures which AXI master to access the source.

- 0: AXI Master 1
- 1: AXI Master 2
- (R/W)

DMAC_CH n _DMS Configures which AXI master to access the destination.

- 0: AXI Master 1
- 1: AXI Master 2
- (R/W)

DMAC_CH n _SINC Configures whether to increment the source address on every source transfer.

- If VDMA is fetching data from a source peripheral's FIFO at a fixed address, then set this field to 1.
- 0: Increment
- 1: No change
- (R/W)

DMAC_CH n _DINC Configures whether to increment the destination address on every destination transfer. If VDMA is writing data to a source peripheral's FIFO at a fixed address, then set this field to 1.

- 0: Increment
- 1: No change
- (R/W)

DMAC_CH n _SRC_TR_WIDTH Configures source transfer width.

- 0x0: 8 bits
- 0x1: 16 bits
- 0x2: 32 bits
- 0x3: 64 bits
- (R/W)

Continued on the next page...

Register 5.11. DMAC_CH n _CTLO_REG (n : 1-4) (0x0100* n + 0x0018)

Continued from the previous page...

DMAC_CH n _DST_TR_WIDTH Configures destination transfer width.

0x0: 8 bits
0x1: 16 bits
0x2: 32 bits
0x3: 64 bits
(R/W)

DMAC_CH n _SRC_MSIZ Configures the length of the source burst transaction. Every time the handshaking interface requests for a source burst transaction, the read value of this field from the source is the number of data items, each of width **DMAC_CH n _SRC_TR_WIDTH**.

0x0: 1 data item
0x1: 4 data items
0x2: 8 data items
0x3: 16 data items
0x4: 32 data items
0x5: 64 data items
0x6: 128 data items
0x7: 256 data items
0x8: 512 data items
0x9: 1024 data items
(R/W)

DMAC_CH n _DST_MSIZ Configures the length of the destination burst transaction. Every time the handshaking interface requests for a destination burst transaction, the read value of this field from the destination is the number of data items, each of width **DMAC_CH n _SRC_TR_WIDTH**.

0x0: 1 data item
0x1: 4 data items
0x2: 8 data items
0x3: 16 data items
0x4: 32 data items
0x5: 64 data items
0x6: 128 data items
0x7: 256 data items
0x8: 512 data items
0x9: 1024 data items
(R/W)

Continued on the next page...

Register 5.11. DMAC_CH n _CTLO_REG (n : 1-4) (0x0100* n + 0x0018)

Continued from the previous page...

DMAC_CH n _NONPOSTED_LASTWRITE_EN Configures whether to enable non-posted writes throughout the block transfer.

Posted writes refers to sending data without the need to write the data into the destination (the data may still be in transit); VDMA can consider this write request to be complete. Non-posted writes refers to sending data that needs to be written into the destination and requires a response, only then can VDMA consider this write request to be complete.

0: Posted writes can be used throughout the block transfer.

1: Posted writes can be used at the end of the block transfer (within the block). The last write must be non-posted to ensure synchronization between the generation of the block completion interrupt and the final written data reaching the destination.

(R/W)

Register 5.12. DMAC_CH n _CTL1_REG (n : 1-4) (0x0100* n + 0x001C)

DMAC_CH _n _SHADOWREG_OR_LLI_VALID																															DMAC_CH _n _SHADOWREG_OR_LLI_LAST																															(reserved)																															DMAC_CH _n _IOC_BLKTRF																															DMAC_CH _n _DST_STAT_EN																															DMAC_CH _n _SRC_STAT_EN																															DMAC_CH _n _AWLEN																															DMAC_CH _n _AWLEN_EN																															DMAC_CH _n _ARLEN																															DMAC_CH _n _ARLEN_EN																															(reserved)																														
31			30			29			27			26			25			24			23			16			15			14			7			6			5			0																																																																																																																																																																																																																																																																																																										
0			0			0			0			0			0			0			0			0x0			0			0x0			0			0			x			0			Reset																																																																																																																																																																																																																																																																																																							

DMAC_CH n _ARLEN_EN Configures whether to enable the length of the source burst transfer.

0: VDMA uses any possible value that is less than or equal to 16 as the length of the source burst transfer.

1: VDMA uses the value of **DMAC_CH n _ARLEN** as the length of the source burst transfer till the extent possible. The remaining transfers use the maximum possible burst transfer length. For example, if the data size is 192 bytes and the source burst transfer length **DMAC_CH n _ARLEN** is set to 128 bytes, then after transmitting 128 bytes, the remaining 64 bytes is the maximum possible burst transfer length.

(R/W)

DMAC_CH n _ARLEN Configures the length of the source burst transfer. The specified burst length is used for source data transfer till the extent possible. The remaining transfers use the maximum possible value that is less than or equal to 16.

(R/W)

DMAC_CH n _AWLEN_EN Configures whether to enable the length of the destination burst transfer.

0: VDMA uses any possible value that is less than or equal to 16 as the length of the destination burst transfer.

1: VDMA uses the value of **DMAC_CH n _AWLEN** as the length of the destination burst transfer till the extent possible. The remaining transfers use the maximum possible burst transfer length.

(R/W)

DMAC_CH n _AWLEN Configures the length of the destination burst transfer. The specified burst length is used for destination data transfer till the extent possible. The remaining transfers use the maximum possible value that is less than or equal to 16.

(R/W)

DMAC_CH n _SRC_STAT_EN Configures whether to enable fetch of source status.

This logic enables the fetch of the status from the source peripheral of channel n pointed to by **DMAC_CH n _SSTATARO**. The value is subsequently stored in **DMAC_CH n _SSTAT**. At the end of each block transfer, the status value is written back to the **DMAC_CH n _SSTAT** register of the linked list, if either source or destination peripheral uses linked-list-based multi-block transfer.

0: Do not fetch source status

1: Fetch source status and store the value in **DMAC_CH n _SSTAT**

(R/W)

Continued on the next page...

Register 5.12. DMAC_CH n _CTL1_REG (n : 1-4) (0x0100* n + 0x001C)

Continued from the previous page...

DMAC_CH n _DST_STAT_EN Configures whether to enable fetch of destination status.

This logic enables the fetch of the status from the destination peripheral of channel n pointed to by [DMAC_CH \$n\$ _DSTATARO](#). The value is subsequently stored in [DMAC_CH \$n\$ _DSTAT](#). At the end of each block transfer, the status value is written back to the [DMAC_CH \$n\$ _DSTAT](#) register of the linked list, if either source or destination peripheral uses linked-list-based multi-block transfer.

0: Do not fetch destination status

1: Fetch destination status and store the value in [DMAC_CH \$n\$ _DSTAT](#)

(R/W)

DMAC_CH n _IOC_BLKTRF Configures whether to enable the interrupt on completion of multi-block transfer based on shadow register or linked list.

Note: If the source and destination uses contiguous-address or auto-reloading-based multi-block transfer, this field cannot be modified on a block by block basis. In addition, the programmed value before enabling the channel will be used for all the blocks in DMA transfer.

0: Disable [DMAC_CH \$n\$ _BLOCK_TFR_DONE_INTSTAT](#)

1: Enable [DMAC_CH \$n\$ _BLOCK_TFR_DONE_INTSTAT](#), valid when

[DMAC_CH \$n\$ _ENABLE_BLOCK_TFR_DONE_INTSTAT](#) is 1. To output a [DMAC_CH \$n\$ _BLOCK_TFR_DONE_INT](#) interrupt, set [DMAC_CH \$n\$ _ENABLE_BLOCK_TFR_DONE_INTSIGNAL](#) to 1.

(R/W)

DMAC_CH n _SHADOWREG_OR_LLI_LAST Configures whether the shadow register or linked list item is the last one.

0: Not the last one

1: The last one

(R/W)

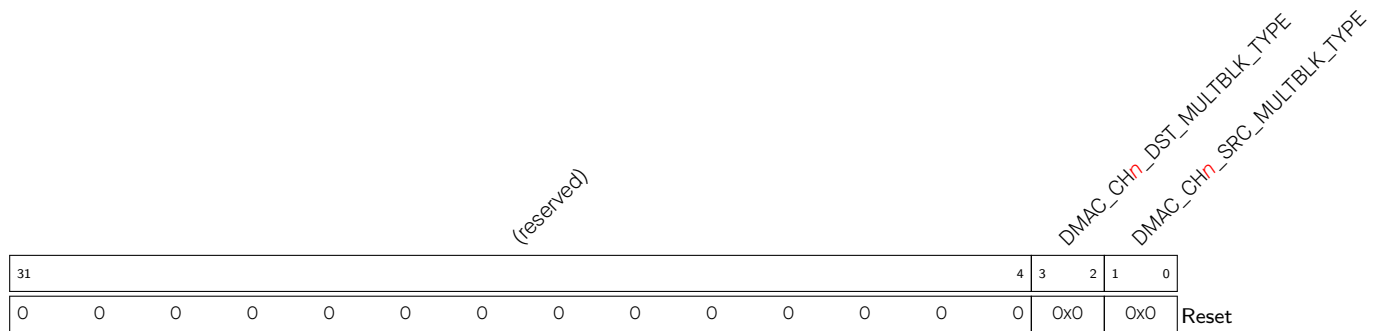
DMAC_CH n _SHADOWREG_OR_LLI_VALID Configures whether the contents of shadow register or the linked list item are valid.

0: Invalid

1: Valid

(R/W)

Register 5.13. DMAC_CH*n*_CFG0_REG (*n*: 1-4) (0x0100n* + 0x0120)**



DMAC_CHn_SRC_MULTBLK_TYPE Configures the source multi-block transfer type.

- 0: Contiguous address
 - 1: Auto reloading
 - 2: Shadow register
 - 3: Linked list
- (R/W)

DMAC_CHn_DST_MULTBLK_TYPE Configures the destination multi-block transfer type.

- 0: Contiguous address
- 1: Auto reloading
- 2: Shadow register
- 3: Linked list

If both `DMAC_CHn_SRC_MULTBLK_TYPE` and `DMAC_CHn_DST_MULTBLK_TYPE` are 0, then the transfer type is single-block transfer based on contiguous address.

(R/W)

Register 5.14. DMAC_CH n _CFG1_REG (n : 1-4) (0x0100* n + 0x0124)

(reserved)		DMAC_CH _n _DST_OSR_LMT				DMAC_CH _n _SRC_OSR_LMT				DMAC_CH _n _LOCK_CH_L				DMAC_CH _n _LOCK_CH				(reserved)		DMAC_CH _n _DST_PER				(reserved)				DMAC_CH _n _SRC_PER				(reserved)		DMAC_CH _n _TT_FC	
31	30	27		26	23		22	21	20	19	17		16	14		13	12	11	9		8	7	6	3				2	0						
0	0x0		0x0		0x0		0x0	0	0x3		0	0	0	0x0	0	0	0	0x0	0	0	0	0x0	0	0	0	0	0x3		Reset						

DMAC_CHn_TT_FC Configures transfer type and flow controller.

0x0: Transfer type is memory to memory and flow controller is VDMA

Ox1: Transfer type is memory to peripheral and flow controller is VDMA

Ox2: Transfer type is peripheral to memory and flow controller is VDMA

0x3: Invalid

0x4: Transfer type is peripheral to memory and flow controller is source peripheral

0x5: Invalid

Ox6: Transfer type is memory to peripheral and flow controller is destination peripheral

0x7: Invalid

(R/W)

DMAC_CH n _SRC_PER Assigns a handshaking interface (0 ~ 2) to the source of channel n . (R/W)

DMAC_CH n _DST_PER Assigns a handshaking interface (0 ~ 2) to the destination of channel n .
(R/W)

DMAC_CHn_CH_PRIOR Assigns channel priority (0 ~ 3). A priority of 3 is the highest priority, and 0 is the lowest. A programmed value outside this range will cause erroneous behavior. (R/W)

DMAC_CH_n_LOCK_CH VDMA does not support lock feature. Reads of this field always return 0.
(RO)

DMAC_CH_n_LOCK_CH_L VDMA does not support lock feature. Reads of this field always return 0.
(RO)

DMAC_CH_n_SRC_OSR_LMT Configures the limit of the source AXI Outstanding request. The maximum number of AXI Outstanding request is 16.

Source AXI Outstanding request limit = Programmed value + 1.

AXI Outstanding is a feature in the AXI protocol that allows to send request before the previous request response is received, thereby increasing the bandwidth of the bus interface.

(R/W)

DMAC_CHn_DST_OSR_LMT Configures the limit of the destination AXI Outstanding request. The maximum number of AXI Outstanding request is 16.

Destination AXI Outstanding request limit = Programmed value + 1.

(R/W)

Register 5.15. DMAC_CH n _LLPO_REG (n : 1-4) (0x0100* n + 0x0128)

DMAC_CH _n _LOC0																(reserved)				DMAC_CH _n _LMS				
31																6	5					1	0	
0x00000																	0	0	0	0	0	0	0	Reset

DMAC_CH n _LMS Selects the AXI master for accessing LLI.

0: AXI Master 1

1: AXI Master 2

(R/W)

DMAC_CH n _LOC0 Configures the starting address of the first linked list item in memory. The six least significant bits (LSB) of the starting address are not stored because it is assumed that the address is 64-byte aligned. (R/W)

Register 5.16. DMAC_CH n _BLK_TFR_RESUMEREQ0_REG (n : 1-4) (0x0100* n + 0x0148)

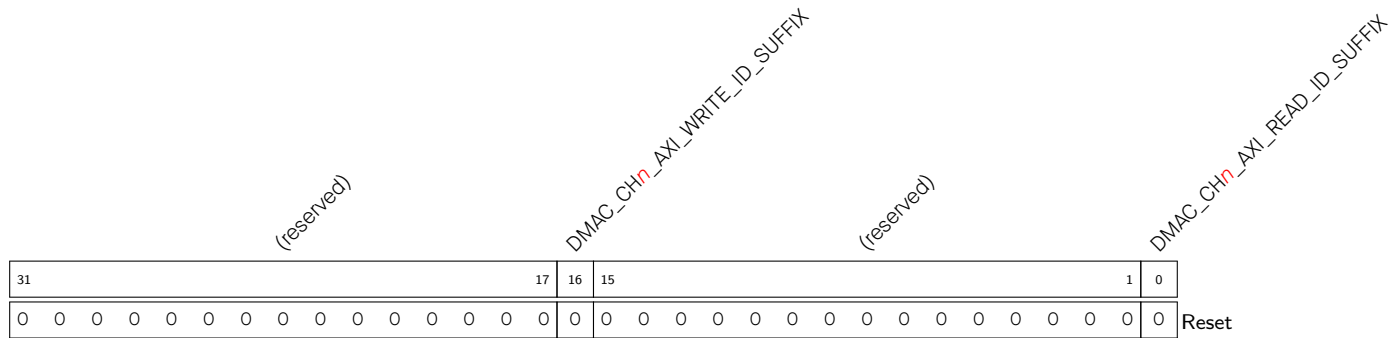
(reserved)																																DMAC_CH _n _BLK_TFR_RESUMEREQ																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
31																															1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0</

DMAC_CH n _BLK_TFR_RESUMEREQ Configures whether to request to resume block transfer during linked-list or shadow-register-based multi-block transfer.

0: Do not request to resume block transfer

1: Request to resume block transfer

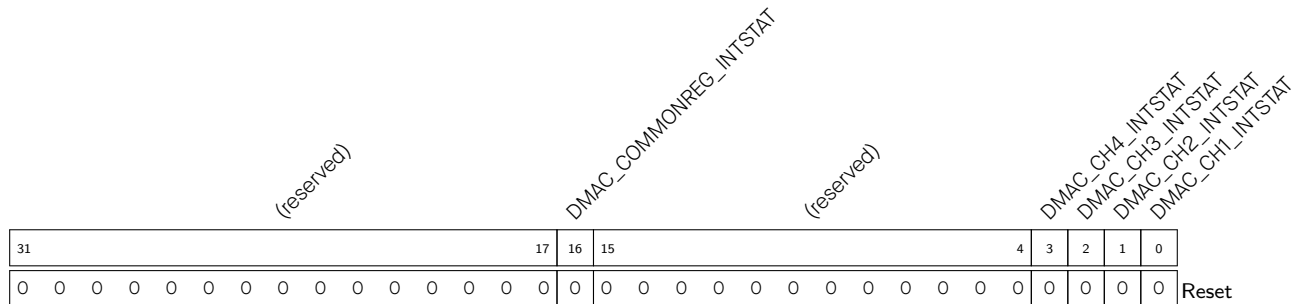
(WO)

Register 5.17. DMAC_CH n _AXI_ID0_REG (0x0150)

DMAC_CH n _AXI_READ_ID_SUFFIX Configure the least significant bit of AXI read ID. (R/W)

DMAC_CH n _AXI_WRITE_ID_SUFFIX Configure the least significant bit of AXI write ID. (R/W)

Register 5.18. DMAC_INTSTATUS0_REG (0x0030)



DMAC_CH n _INTSTAT (n : 1-4) Indicates the interrupt status for channel n .

- 1: Interrupt is active
 - 0: Interrupt is inactive
- (RO)

DMAC_COMMONREG_INTSTAT Indicates the common register interrupt status.

- 1: Interrupt is active
 - 0: Interrupt is inactive
- (RO)

Register 5.19. DMAC_COMMONREG_INTCLEAR0_REG (0x0038)

(reserved)																DMAC_CLEAR_SLVIF_UNDEFINEDREG_DEC_ERR_INTSTAT				(reserved)				DMAC_CLEAR_SLVIF_COMMONREG_RD2WO_ERR_INTSTAT				DMAC_CLEAR_SLVIF_COMMONREG_WR2RO_ERR_INTSTAT				DMAC_CLEAR_SLVIF_COMMONREG_DEC_ERR_INTSTAT							
31																	9	8	7					3	2	1	0												
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset						
DMAC_CLEAR_SLVIF_COMMONREG_DEC_ERR_INTSTAT																Write		1		to		clear				DMAC_SLVIF_COMMONREG_DEC_ERR_INTSTAT. (WO)													
DMAC_CLEAR_SLVIF_COMMONREG_WR2RO_ERR_INTSTAT																Write		1		to		clear				DMAC_SLVIF_COMMONREG_WR2RO_ERR_INTSTAT. (WO)													
DMAC_CLEAR_SLVIF_COMMONREG_RD2WO_ERR_INTSTAT																Write		1		to		clear				DMAC_SLVIF_COMMONREG_RD2WO_ERR_INTSTAT. (WO)													
DMAC_CLEAR_SLVIF_UNDEFINEDREG_DEC_ERR_INTSTAT																Write		1		to		clear				DMAC_SLVIF_UNDEFINEDREG_DEC_ERR_INTSTAT. (WO)													

Register 5.20. DMAC_COMMONREG_INTSTATUS_ENABLE0_REG (0x0040)

(reserved)																												DMAC_ENABLE_SLVIF_UNDEFINEDREG_DEC_ERR_INTSTAT				(reserved)				DMAC_ENABLE_SLVIF_COMMONREG_RD2WO_ERR_INTSTAT				DMAC_ENABLE_SLVIF_COMMONREG_WR2RO_ERR_INTSTAT				DMAC_ENABLE_SLVIF_COMMONREG_DEC_ERR_INTSTAT			
31																												9	8	7	3				2	1	0	Reset									
0 0																												1	0 0 0 0 0 0				1	1	1												

- DMAC_ENABLE_SLVIF_COMMONREG_DEC_ERR_INTSTAT

Write

1

to

enable

DMAC_SLVIF_COMMONREG_DEC_ERR_INTSTAT. (R/W)
- DMAC_ENABLE_SLVIF_COMMONREG_WR2RO_ERR_INTSTAT

Write

1

to

enable

DMAC_SLVIF_COMMONREG_WR2RO_ERR_INTSTAT. (R/W)
- DMAC_ENABLE_SLVIF_COMMONREG_RD2WO_ERR_INTSTAT

Write

1

to

enable

DMAC_SLVIF_COMMONREG_RD2WO_ERR_INTSTAT. (R/W)
- DMAC_ENABLE_SLVIF_UNDEFINEDREG_DEC_ERR_INTSTAT

Write

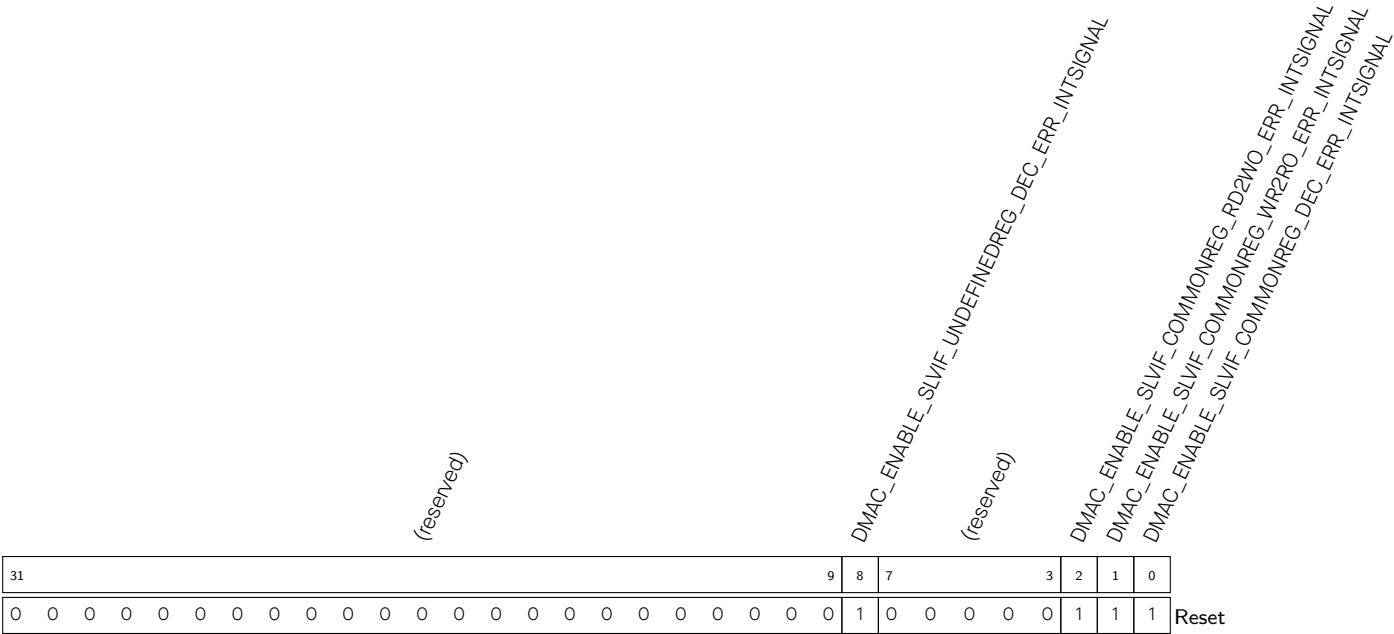
1

to

enable

DMAC_SLVIF_UNDEFINEDREG_DEC_ERR_INTSTAT. (R/W)

Register 5.21. DMAC_COMMONREG_INTSIGNAL_ENABLE0_REG (0x0048)



- DMAC_ENABLE_SLVIF_COMMONREG_DEC_ERR_INTSIGNAL

Write

1

to

enable

DMAC_SLVIF_COMMONREG_DEC_ERR_INT at port level. (R/W)
- DMAC_ENABLE_SLVIF_COMMONREG_WR2RO_ERR_INTSIGNAL

Write

1

to

enable

DMAC_SLVIF_COMMONREG_WR2RO_ERR_INT at port level. (R/W)
- DMAC_ENABLE_SLVIF_COMMONREG_RD2WO_ERR_INTSIGNAL

Write

1

to

enable

DMAC_SLVIF_COMMONREG_RD2WO_ERR_INT at port level. (R/W)
- DMAC_ENABLE_SLVIF_UNDEFINEDREG_DEC_ERR_INTSIGNAL

Write

1

to

enable

DMAC_SLVIF_UNDEFINEDREG_DEC_ERR_INT at port level. (R/W)

Register 5.22. DMAC_COMMONREG_INTSTATUSO_REG (0x0050)

(reserved)																								DMAC_SLVIF_UNDEFINEDREG_DEC_ERR_INTSTAT				(reserved)				DMAC_SLVIF_COMMONREG_RD2WO_ERR_INTSTAT				DMAC_SLVIF_COMMONREG_WR2RO_ERR_INTSTAT				DMAC_SLVIF_COMMONREG_DEC_ERR_INTSTAT																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
31																								9	8	7	3				2	1	0	Reset																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

DMAC_SLVIF_COMMONREG_DEC_ERR_INTSTAT Slave interface common register decode error interrupt status bit. This error occurs when VDMA accesses an invalid address in the common register space (0x000 to 0xFF).

0: No error occurs

1: Error occurs

(RO)

DMAC_SLVIF_COMMONREG_WR2RO_ERR_INTSTAT Slave interface common register write to read-only error interrupt status bit. This error occurs when a write operation is performed to a read-only register in the common register space (0x000 to 0xFF).

0: No error occurs

1: Error occurs

(RO)

DMAC_SLVIF_COMMONREG_RD2WO_ERR_INTSTAT Slave interface common register read to write-only error interrupt status bit. This error occurs when a read operation is performed to a write-only register in the common register space (0x000 to 0xFF).

0: No error occurs

1: Error occurs

(RO)

DMAC_SLVIF_UNDEFINEDREG_DEC_ERR_INTSTAT Slave interface undefined register decode error interrupt signal enable bit. This error occurs when the register access is to undefined address range (>0x4FF).

0: No error occurs

1: Error occurs

(RO)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	0	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1
Reset																															

DMAC_CHn_ENABLE_LLI_WR_DEC_ERR_INTSTAT Write 1 to enable
DMAC_CHn_LLI_WR_DEC_ERR_INTSTAT. (R/W)

Register 5.23. DMAC_CH n _INTSTATUS_ENABLE0_REG (n : 1-4) (0x0100* n + 0x0180)

Continued from the previous page...

DMAC_CH n _ENABLE_LLI_RD_SLV_ERR_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _LLI_RD_SLV_ERR_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_LLI_WR_SLV_ERR_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _LLI_WR_SLV_ERR_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_SHADOWREG_OR_LLI_INVALID_ERR_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _SHADOWREG_OR_LLI_INVALID_ERR_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_SLVIF_MULTIBLKTYPE_ERR_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _SLVIF_MULTIBLKTYPE_ERR_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_SLVIF_DEC_ERR_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _SLVIF_DEC_ERR_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_SLVIF_WR2RO_ERR_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _SLVIF_WR2RO_ERR_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_SLVIF_RD2RWO_ERR_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _SLVIF_RD2RWO_ERR_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_SLVIF_WRONCHEN_ERR_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _SLVIF_WRONCHEN_ERR_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_SLVIF_SHADOWREG_WRON_VALID_ERR_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _SLVIF_SHADOWREG_WRON_VALID_ERR_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_CH_SRC_SUSPENDED_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _CH_SRC_SUSPENDED_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_CH_SUSPENDED_INTSTAT Write 1 to enable
[DMAC_CH \$n\$ _CH_SUSPENDED_INTSTAT](#). (R/W)

DMAC_CH n _ENABLE_CH_DISABLED_INTSTAT Write 1 to enable [DMAC_CH \$n\$ _CH_DISABLED_INTSTAT](#).
 (R/W)

DMAC_CH n _ENABLE_CH_ABORTED_INTSTAT Write 1 to enable [DMAC_CH \$n\$ _CH_ABORTED_INTSTAT](#).
 (R/W)

Register 5.24. DMAC_CH n _INTSTATUSO_REG (n : 1-4) (0x0100* n + 0x0188)

DMAC_CHn_CH_ABORTED_INTSTAT DMAC_CHn_CH_DISABLED_INTSTAT DMAC_CHn_CH_SUSPENDED_INTSTAT DMAC_CHn_CH_SRC_SUSPENDED_INTSTAT (reserved) DMAC_CHn_SLVIF_SHADOWREG_WRON_VALID_ERR_INTSTAT DMAC_CHn_SLVIF_WRONCHEN_ERR_INTSTAT DMAC_CHn_SLVIF_RD2RWO_ERR_INTSTAT DMAC_CHn_SLVIF_WR2RO_ERR_INTSTAT (reserved) DMAC_CHn_SLVIF_DEC_ERR_INTSTAT DMAC_CHn_SLVIF_MULTIBLKTYPE_ERR_INTSTAT DMAC_CHn_SHADOWREG_OR_LLI_WR_SLV_ERR_INTSTAT DMAC_CHn_LLI_WR_SLV_ERR_INTSTAT DMAC_CHn_LLI_RD_SLV_ERR_INTSTAT DMAC_CHn_LLI_WR_DEC_ERR_INTSTAT DMAC_CHn_DST_SLV_ERR_INTSTAT DMAC_CHn_SRC_SLV_ERR_INTSTAT DMAC_CHn_DST_DEC_ERR_INTSTAT (reserved) DMAC_CHn_SRC_TRANSCOMP_INTSTAT DMAC_CHn_DMA_TFR_DONE_INTSTAT DMAC_CHn_BLOCK_TFR_DONE_INTSTAT																																	
31	30	29	28	27							21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

DMAC_CH n _BLOCK_TFR_DONE_INTSTAT Block transfer done status bit. (RO)

DMAC_CH n _DMA_TFR_DONE_INTSTAT DMA transfer done status bit. (RO)

DMAC_CH n _SRC_TRANSCOMP_INTSTAT Source DMA transfer completed status bit. (RO)

DMAC_CH n _DST_TRANSCOMP_INTSTAT Destination DMA transfer completed status bit. (RO)

DMAC_CH n _SRC_DEC_ERR_INTSTAT Source decode error status bit. The decode error is detected by the master interface during source transfer. (RO)

DMAC_CH n _DST_DEC_ERR_INTSTAT Destination decode error status bit. The decode error is detected by the master interface during destination transfer. (RO)

DMAC_CH n _SRC_SLV_ERR_INTSTAT Source slave error status bit. The slave error is detected by the master interface during source transfer. This error occurs if the slave interface that reads data issues a slave error. (RO)

DMAC_CH n _DST_SLV_ERR_INTSTAT Destination slave error status bit. The slave error is detected by the master interface during source transfer. This error occurs if the slave interface that writes data issues a slave error. (RO)

DMAC_CH n _LLI_RD_DEC_ERR_INTSTAT LLI read decode error status bit. The decode error is detected by the master interface during LLI read operation. (RO)

DMAC_CH n _LLI_WR_DEC_ERR_INTSTAT LLI write decode error status bit. The decode error is detected by the master interface during LLI write-back operation. (RO)

DMAC_CH n _LLI_RD_SLV_ERR_INTSTAT LLI read slave error status bit. The slave error is detected by the master interface during LLI read operation. (RO)

DMAC_CH n _LLI_WR_SLV_ERR_INTSTAT LLI write slave error status bit. The slave error is detected by the master interface during LLI write-back operation. (RO)

Continued on the next page...

Register 5.24. DMAC_CH n _INTSTATUSO_REG (n : 1-4) (0x0100* n + 0x0188)

Continued from the previous page...

DMAC_CH n _SHADOWREG_OR_LLI_INVALID_ERR_INTSTAT Shadow register or LLI invalid error status bit. This error occurs if **DMAC_CH n _SHADOWREG_OR_LLI_VALID** is 0 during VDMA shadow register/LLI fetch phase. (RO)

DMAC_CH n _SLVIF_MULTIBLKTYPE_ERR_INTSTAT Slave interface multi-block type error status bit. This error occurs if multi-block transfer type programmed in **DMAC_CH n _SRC_MULTBLK_TYPE** and **DMAC_CH n _DST_MULTBLK_TYPE** is invalid. (RO)

DMAC_CH n _SLVIF_DEC_ERR_INTSTAT Slave interface decode error status bit. The decode error is generated by VDMA during register access. This error occurs if the register access is to invalid address in channel n . (RO)

DMAC_CH n _SLVIF_WR2RO_ERR_INTSTAT Slave interface write to read-only error status bit. This error occurs if write operation is performed to a read-only register. (RO)

DMAC_CH n _SLVIF_RD2RWO_ERR_INTSTAT Slave interface read to write-only error status bit. This error occurs if read operation is performed to a write-only register. (RO)

DMAC_CH n _SLVIF_WRONCHEN_ERR_INTSTAT Slave interface write to enable channel error status bit. This error occurs if an illegal write operation is performed on a register as per the VDMA specification. (RO)

DMAC_CH n _SLVIF_SHADOWREG_WRON_VALID_ERR_INTSTAT Shadow register write on valid error status bit. This error occurs if shadow register based multi-block transfer is enabled and software tries to write to the shadow register when **DMAC_CH n _SHADOWREG_OR_LLI_VALID** bit is 1. (RO)

DMAC_CH n _CH_SRC_SUSPENDED_INTSTAT Channel source suspended status bit.
 0: Channel source is not suspended.
 1: Channel Source is suspended.
 (RO)

DMAC_CH n _CH_SUSPENDED_INTSTAT Channel suspended status bit.
 0: Channel is not suspended.
 1: Channel is suspended.
 (RO)

DMAC_CH n _CH_DISABLED_INTSTAT Channel disabled status bit.
 0: Channel is not disabled.
 1: Channel is disabled.
 (RO)

DMAC_CH n _CH_ABORTED_INTSTAT Channel aborted status bit.
 0: Channel is not aborted.
 1: Channel is aborted.
 (RO)

31	30	29	28	27		21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	Reset
DMAC_CH_n_ENABLE_CH_ABORTED_INTSIGNAL DMAC_CH_n_ENABLE_CH_DISABLED_INTSIGNAL DMAC_CH_n_ENABLE_CH_SUSPENDED_INTSIGNAL DMAC_CH_n_ENABLE_CH_SRC_SUSPENDED_INTSIGNAL (reserved) DMAC_CH_n_ENABLE_SLVIF_SHADOWREG_WRON_VALID_ERR_INTSIGNAL DMAC_CH_n_ENABLE_SLVIF_WRONCHEN_ERR_INTSIGNAL DMAC_CH_n_ENABLE_SLVIF_RD2RWO_ERR_INTSIGNAL DMAC_CH_n_ENABLE_SLVIF_WR2RO_ERR_INTSIGNAL (reserved) DMAC_CH_n_ENABLE_SLVIF_DEC_ERR_INTSIGNAL DMAC_CH_n_ENABLE_SLVIF_DEC_ERR_INTSIGNAL DMAC_CH_n_ENABLE_SLVIF_MULTIBLKTYPE_ERR_INTSIGNAL DMAC_CH_n_ENABLE_SHADOWREG_OR_LLI_INVALID_ERR_INTSIGNAL DMAC_CH_n_ENABLE_LLI_WR_SLV_ERR_INTSIGNAL DMAC_CH_n_ENABLE_LLI_RD_SLV_ERR_INTSIGNAL DMAC_CH_n_ENABLE_LLI_WR_DEC_ERR_INTSIGNAL DMAC_CH_n_ENABLE_LLI_RD_DEC_ERR_INTSIGNAL DMAC_CH_n_ENABLE_DST_SLV_ERR_INTSIGNAL DMAC_CH_n_ENABLE_DST_SLV_ERR_INTSIGNAL DMAC_CH_n_ENABLE_DST_DEC_ERR_INTSIGNAL DMAC_CH_n_ENABLE_DST_DEC_ERR_INTSIGNAL (reserved) DMAC_CH_n_ENABLE_SRC_TRANSCOMP_INTSIGNAL DMAC_CH_n_ENABLE_DMA_TFR_DONE_INTSIGNAL DMAC_CH_n_ENABLE_BLOCK_TFR_DONE_INTSIGNAL																												

DMAC_CHn_ENABLE_LLI_WR_DEC_ERR_INTSIGNAL Write 1 to enable **DMAC_CHn_LLI_WR_DEC_ERR_INT** at port level. (R/W)

ESP32-P4 TRM
PRELIMINARY

Register 5.25. DMAC_CH n _INTSIGNAL_ENABLE0_REG (n : 1-4) (0x0100* n + 0x0190)

Continued from the previous page...

DMAC_CH n _ENABLE_LLI_RD_SLV_ERR_INTSIGNAL	Write	1	to	enable
DMAC_CH n _LLI_RD_SLV_ERR_INT at port level. (R/W)				
DMAC_CH n _ENABLE_LLI_WR_SLV_ERR_INTSIGNAL	Write	1	to	enable
DMAC_CH n _LLI_WR_SLV_ERR_INT at port level. (R/W)				
DMAC_CH n _ENABLE_SHADOWREG_OR_LLI_INVALID_ERR_INTSIGNAL	Write	1	to	enable
DMAC_CH n _SHADOWREG_OR_LLI_INVALID_ERR_INT at port level. (R/W)				
DMAC_CH n _ENABLE_SLVIF_MULTIBLKTYPE_ERR_INTSIGNAL	Write	1	to	enable
DMAC_CH n _SLVIF_MULTIBLKTYPE_ERR_INT at port level. (R/W)				
DMAC_CH n _ENABLE_SLVIF_DEC_ERR_INTSIGNAL	Write	1	to	enable
DMAC_CH n _SLVIF_DEC_ERR_INT at port level. (R/W)				
DMAC_CH n _ENABLE_SLVIF_WR2RO_ERR_INTSIGNAL	Write	1	to	enable
DMAC_CH n _SLVIF_WR2RO_ERR_INT at port level. (R/W)				
DMAC_CH n _ENABLE_SLVIF_RD2RWO_ERR_INTSIGNAL	Write	1	to	enable
DMAC_CH n _SLVIF_RD2RWO_ERR_INT at port level. (R/W)				
DMAC_CH n _ENABLE_SLVIF_WRONCHEN_ERR_INTSIGNAL	Write	1	to	enable
DMAC_CH n _SLVIF_WRONCHEN_ERR_INT at port level. (R/W)				
DMAC_CH n _ENABLE_SLVIF_SHADOWREG_WRON_VALID_ERR_INTSIGNAL	Write	1	to	enable
DMAC_CH n _SLVIF_SHADOWREG_WRON_VALID_ERR_INT at port level. (R/W)				
DMAC_CH n _ENABLE_CH_SRC_SUSPENDED_INTSIGNAL	Write	1	to	enable
DMAC_CH n _CH_SRC_SUSPENDED_INT at port level. (R/W)				
DMAC_CH n _ENABLE_CH_SUSPENDED_INTSIGNAL	Write	1	to	enable
DMAC_CH n _CH_SUSPENDED_INT at port level. (R/W)				
DMAC_CH n _ENABLE_CH_DISABLED_INTSIGNAL	Write	1	to	enable
DMAC_CH n _CH_DISABLED_INT at port level. (R/W)				
DMAC_CH n _ENABLE_CH_ABORTED_INTSIGNAL	Write 1 to enable DMAC_CH n _CH_ABORTED_INT at port level. (R/W)			

Register 5.26. DMAC_CH n _INTCLEARO_REG (n : 1-4) (0x0100* n + 0x0198)

[illegible]

DMAC_CH*n*_CLEAR_BLOCK_TFR_DONE_INTSTAT Write 1 to clear
DMAC_CH*n*_BLOCK_TFR_DONE_INTSTAT. (WO)

DMAC_CH n _CLEAR_DMA_TFR_DONE_INTSTAT Write 1 to clear DMAC_CH n _DMA_TFR_DONE_INTSTAT.
(WO)

DMAC_CH n _CLEAR_SRC_TRANSCOMP_INTSTAT Write 1 to clear **DMAC_CH n _SRC_TRANSCOMP_INTSTAT**.
(WO)

DMAC_CH n _CLEAR_DST_TRANSCOMP_INTSTAT Write 1 to clear **DMAC_CH n _DST_TRANSCOMP_INTSTAT**.
(WO)

DMAC_CH n _CLEAR_SRC_DEC_ERR_INTSTAT Write 1 to clear **DMAC_CH n _SRC_DEC_ERR_INTSTAT**.
(WO)

DMAC_CHn_CLEAR_DST_DEC_ERR_INTSTAT Write 1 to clear **DMAC_CHn_DST_DEC_ERR_INTSTAT**.
(WO)

DMAC_CH n _CLEAR_SRC_SLV_ERR_INTSTAT Write 1 to clear **DMAC_CH n _SRC_SLV_ERR_INTSTAT**.
(WO)

DMAC_CH n _CLEAR_DST_SLV_ERR_INTSTAT Write 1 to clear **DMAC_CH n _DST_SLV_ERR_INTSTAT**.
(WO)

DMAC_CHn_CLEAR_LLI_RD_DEC_ERR_INTSTAT Write 1 to clear DMAC_CHn_LLI_RD_DEC_ERR_INTSTAT.
(WO)

DMAC_CHn_CLEAR_LLI_WR_DEC_ERR_INTSTAT Write 1 to clear DMAC_CHn_LLI_WR_DEC_ERR_INTSTAT.
(WO)

Continued on the next page...

Register 5.26. DMAC_CH n _INTCLEAR0_REG (n : 1-4) (0x0100* n + 0x0198)

Continued from the previous page...

DMAC_CH n _CLEAR_LLI_RD_SLV_ERR_INTSTAT Write 1 to clear DMAC_CH n _LLI_RD_SLV_ERR_INTSTAT.
(WO)

DMAC_CH n _CLEAR_LLI_WR_SLV_ERR_INTSTAT Write 1 to clear DMAC_CH n _LLI_WR_SLV_ERR_INTSTAT.
(WO)

DMAC_CH n _CLEAR_SHADOWREG_OR_LLI_INVALID_ERR_INTSTAT Write 1 to clear
DMAC_CH n _SHADOWREG_OR_LLI_INVALID_ERR_INTSTAT. (WO)

DMAC_CH n _CLEAR_SLVIF_MULTIBLKTYPE_ERR_INTSTAT Write 1 to clear
DMAC_CH n _SLVIF_MULTIBLKTYPE_ERR_INTSTAT. (WO)

DMAC_CH n _CLEAR_SLVIF_DEC_ERR_INTSTAT Write 1 to clear DMAC_CH n _SLVIF_DEC_ERR_INTSTAT.
(WO)

DMAC_CH n _CLEAR_SLVIF_WR2RO_ERR_INTSTAT Write 1 to clear
DMAC_CH n _SLVIF_WR2RO_ERR_INTSTAT. (WO)

DMAC_CH n _CLEAR_SLVIF_RD2RWO_ERR_INTSTAT Write 1 to clear
DMAC_CH n _SLVIF_RD2RWO_ERR_INTSTAT. (WO)

DMAC_CH n _CLEAR_SLVIF_WRONCHEN_ERR_INTSTAT Write 1 to clear
DMAC_CH n _SLVIF_WRONCHEN_ERR_INTSTAT. (WO)

DMAC_CH n _CLEAR_SLVIF_SHADOWREG_WRON_VALID_ERR_INTSTAT Write 1 to clear
DMAC_CH n _SLVIF_SHADOWREG_WRON_VALID_ERR_INTSTAT. (WO)

DMAC_CH n _CLEAR_CH_SRC_SUSPENDED_INTSTAT Write 1 to clear
DMAC_CH n _CH_SRC_SUSPENDED_INTSTAT. (WO)

DMAC_CH n _CLEAR_CH_SUSPENDED_INTSTAT Write 1 to clear DMAC_CH n _CH_SUSPENDED_INTSTAT.
(WO)

DMAC_CH n _CLEAR_CH_DISABLED_INTSTAT Write 1 to clear DMAC_CH n _CH_DISABLED_INTSTAT.
(WO)

DMAC_CH n _CLEAR_CH_ABORTED_INTSTAT Write 1 to clear DMAC_CH n _CH_ABORTED_INTSTAT.
(WO)

Register 5.27. DMAC_CH n _STATUS0_REG (n : 1-4) (0x0100* n + 0x0130)

(reserved)										DMAC_CH _n _CMPLTD_BLK_TFR_SIZE																					
31										22	21																				0
0	0	0	0	0	0	0	0	0	0	0x0000																					Reset

DMAC_CH n _CMPLTD_BLK_TFR_SIZE Indicates the total number of data of width **DMAC_CH n _SRC_TR_WIDTH** transferred during the previous block transfer.

If any error occurs during DMA transfer, the block transfer might be terminated early. In such case, the value of this field indicates the actual data transferred without error in the current block.

This field is cleared to 0 when the channel is enabled.

(RO)

Register 5.28. DMAC_CH n _STATUS1_REG (n : 1-4) (0x0100* n + 0x0134)

(reserved)																DMAC_CH _n _DATA_LEFT_IN_FIFO																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
31																15																14																0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0																0</															

DMAC_CH n _DATA_LEFT_IN_FIFO Indicates the total number of remaining data in the VDMA channel FIFO after completing the current block transfer.

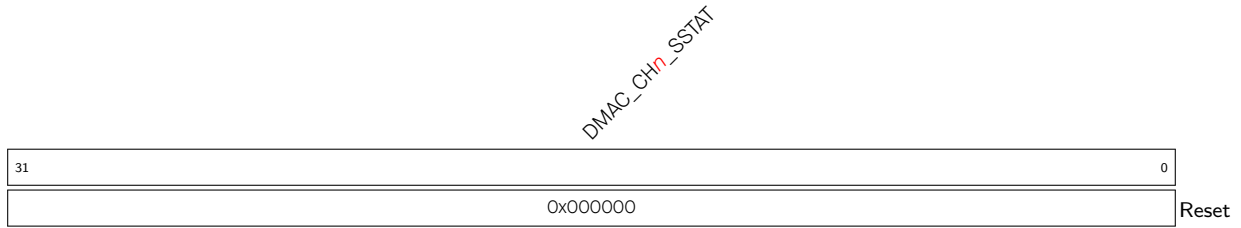
The data width in the channel FIFO is **DMAC_CH n _SRC_TR_WIDTH**.

For a normal block transfer completion without errors, the value of this field is 0. If any errors occur during DMA transfer, the block transfer may be terminated early; in this case, this field indicates the remaining data in the channel FIFO that cannot be transferred to the target peripheral.

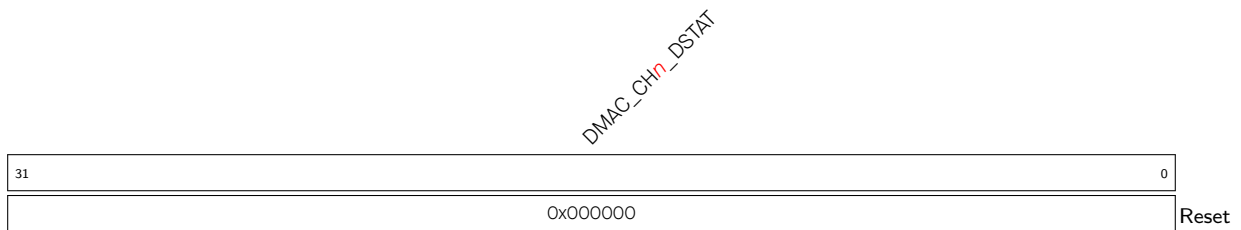
This field will be cleared when the channel is enabled.

If **DMAC_CH n _DST_TR_WIDTH** > **DMAC_CH n _SRC_TR_WIDTH**, there may be remaining data in the FIFO that is not enough to form a single transfer of **DMAC_CH n _SRC_TR_WIDTH** width; in this case, this field will return 0.

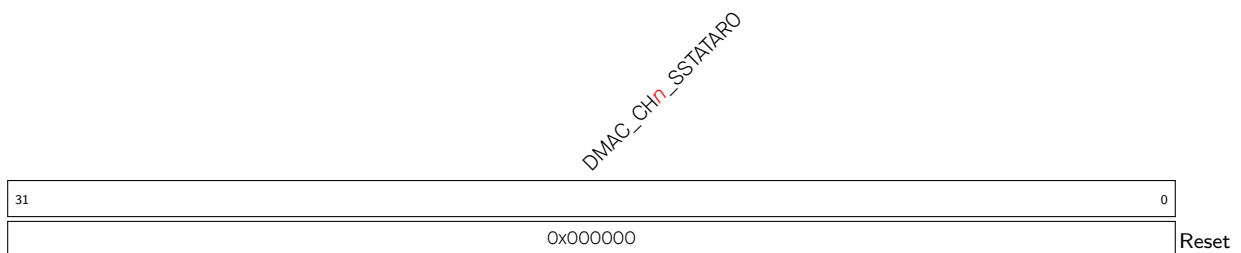
(RO)

Register 5.29. DMAC_CH n _SSTATO_REG (n : 1-4) (0x0100* n + 0x0160)

DMAC_CH n _SSTAT Indicates the source status information retrieved by hardware from the address pointed to by [DMAC_CH \$n\$ _SSTATARO_REG](#). (RO)

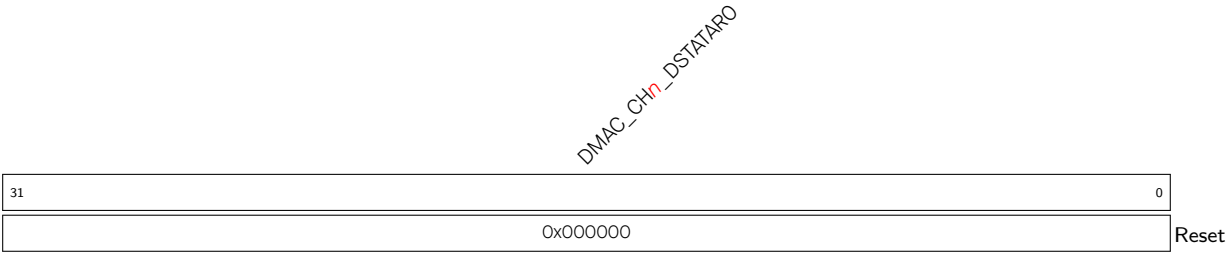
Register 5.30. DMAC_CH n _DSTATO_REG (n : 1-4) (0x0100* n + 0x0168)

DMAC_CH n _DSTAT Indicates the destination status information retrieved by hardware from the address pointed to by [DMAC_CH \$n\$ _DSTATARO_REG](#). (RO)

Register 5.31. DMAC_CH n _SSTATARO_REG (n : 1-4) (0x0100* n + 0x0170)

DMAC_CH n _SSTATARO Configures the address from where the hardware obtains the source status information. (R/W)

Register 5.32. DMAC_CHn_DSTATARO_REG (n: 1-4) (0x0100*n + 0x0178)



DMAC_CHn_DSTATARO Configures the address from where the hardware obtains the destination status information. (R/W)